Real Estate Platform

Architettura serverless su AWS

Relazione di progetto — Servizi Cloud

ITS ICT Piemonte — Software Development

Gruppo
Charrier, Fatticcioni, Badaoui, Pappalardo, Xu, Jimenez

Regione AWS: eu-south-1 (Milano)
Ottobre 2026

Documentazione completa, configurazioni e diagrammi:
https://github.com/MatteoCharrierGit/real-estate-platform

Indice

1. Il problema ... 3
1.1 Che cosa deve fare la piattaforma ... 3
1.2 Perché è un problema da cloud ... 3
1.3 I requisiti ... 4
2. La soluzione in sintesi ... 5
2.1 L'idea e il diagramma ... 5
2.2 Dai requisiti alla soluzione ... 6
2.3 Le scelte di fondo ... 7
3. Come comunicano i componenti ... 8
3.1 Scaricare l'applicazione ... 8
3.2 Login e chiamate all'API ... 11
3.3 Dalla Lambda al database ... 14
3.4 Caricare e servire i file ... 17
3.5 Le notifiche email e la dead-letter queue ... 22
3.6 Gestione e cancellazione dell'account ... 26
3.7 Log, metriche e allarmi ... 29
4. Sicurezza e IAM ... 31
4.1 Le tre facce di IAM ... 31
4.2 La rete come secondo livello ... 32
4.3 La mappa delle autorizzazioni ... 33
5. Decisioni, alternative e limiti ... 35
5.1 Le decisioni principali ... 35
5.2 Limiti e rischi dichiarati ... 36
5.3 Sviluppi possibili ... 36

1. Il problema

Real Estate Platform è il progetto di gruppo per l'esame di Servizi Cloud dell'ITS: una piattaforma immobiliare su AWS. Prima dei servizi, diciamo che cosa deve fare, per chi e con quali vincoli; il capitolo 2 riprende i requisiti uno per uno.

> **Il materiale nel repository**
> Le configurazioni (CORS, coda e DLQ, allarmi, Cognito, condizioni del presigned POST…), la documentazione di ogni servizio e i sorgenti dei diagrammi sono nel repository del progetto: https://github.com/MatteoCharrierGit/real-estate-platform.
>
> Nel testo compaiono per intero solo le policy IAM, ognuna nel paragrafo in cui viene usata.

1.1 Che cosa deve fare la piattaforma

Gli agenti immobiliari pubblicano immobili in vendita. Ogni annuncio ha una descrizione, un prezzo e soprattutto **foto e video**. Chiunque deve poter sfogliare gli annunci senza registrarsi.

Chi è interessato a un immobile si registra e fa un'**offerta**. L'agente che ha pubblicato l'annuncio la vede, la accetta o la rifiuta; chi ha fatto l'offerta può ritirarla finché è in sospeso. A ogni passaggio la persona interessata riceve un'**email**: l'agente quando arriva una nuova offerta, l'acquirente quando conosce l'esito. Ogni utente gestisce il proprio profilo e può cancellare l'account: le offerte in sospeso che lo riguardano vanno chiuse, e l'altra parte deve saperlo.

Gli utenti registrati hanno uno di tre ruoli, e il ruolo decide che cosa possono fare.

Ruolo	Chi è	Che cosa può fare
visitatore	chiunque, senza account	consultare gli annunci, le foto e i video
`buyer`	acquirente	fare offerte e ritirarle, ricevere l'esito per email
`agent`	agente immobiliare	pubblicare, modificare e cancellare i propri immobili, caricarne foto e video, accettare o rifiutare le offerte ricevute, essere avvisato delle nuove offerte
`admin`	amministratore	modificare e cancellare gli immobili di chiunque

1.2 Perché è un problema da cloud

Fin qui è un'applicazione web come tante. Sono i vincoli a rendere il problema interessante: da loro nascono quasi tutte le scelte.

Un carico variabile e imprevedibile

Per giorni può non entrare quasi nessuno; poi un annuncio condiviso bene porta molte visite insieme. Dimensionare un server per il picco vuol dire pagarlo vuoto quasi sempre; dimensionarlo per la media vuol dire cadere proprio quando conta. Serve un sistema che **cresca e si riduca da solo**, fino a zero.

File grandi

Una foto arriva a qualche megabyte, un video a centinaia. Farli passare dal backend vuol dire occupare a lungo il calcolo e scontrarsi con i limiti di dimensione delle richieste. I file vanno caricati e serviti **senza passare dal codice dell'applicazione**, e chi carica non deve poter scrivere dove vuole o caricare qualunque cosa.

Dati personali da proteggere

La piattaforma conosce indirizzi email e offerte economiche: dati personali, soggetti al GDPR. Meno copie esistono di un dato, meno posti ci sono da proteggere e da ripulire quando l'utente chiede di essere cancellato. Il database non deve essere raggiungibile da Internet e ogni componente deve poter fare **solo ciò che gli serve**. Per lo stesso motivo i dati devono restare in Italia.

Costi bassi, senza utenti fissi

Senza utenti fissi, ogni costo che corre anche a sistema vuoto pesa: il modello giusto è **pagare in proporzione all'uso** e tenere al minimo ciò che resta acceso.

Nessun server da gestire

Non vogliamo installare sistemi operativi, applicare patch o sostituire macchine guaste: il lavoro deve stare sull'applicazione e sui collegamenti tra le parti. Tutto ciò che si può, lo gestisce AWS.

Disponibile anche se cade una zona

La piattaforma deve continuare a funzionare, senza perdere dati, se una zona di disponibilità non risponde. E un'email il cui invio fallisce non deve sparire in silenzio: va ritentata o messa da parte perché qualcuno la veda.

1.3 I requisiti

I primi sette requisiti sono funzionali, gli altri non funzionali.

- **R1 — Annunci pubblici.** Chiunque consulta gli annunci, con foto e video, senza account.
- **R2 — Registrazione, login e ruoli.** Gli utenti si registrano e accedono; ognuno ha un ruolo tra `buyer`, `agent` e `admin`, che decide che cosa può fare.
- **R3 — Gestione degli immobili.** Un `agent` pubblica, modifica e cancella i propri immobili; un `admin` modifica e cancella quelli di chiunque.
- **R4 — Foto e video.** Un `agent` carica i file dei propri annunci, con limiti di tipo e dimensione; i file sono serviti a tutti in modo veloce.
- **R5 — Offerte.** Un `buyer` fa e ritira offerte; l'`agent` proprietario dell'immobile le accetta o le rifiuta.
- **R6 — Notifiche.** Chi è coinvolto in un'offerta riceve un'email a ogni passaggio, e un nuovo utente riceve il benvenuto.
- **R7 — Account.** Ogni utente gestisce il proprio profilo e può cancellare l'account; le offerte in sospeso che lo riguardano si chiudono e l'altra parte viene avvisata.
- **R8 — Scalabilità e costo proporzionale.** Il sistema regge picchi improvvisi, scala fino a zero e costa in proporzione all'uso.
- **R9 — Nessun server da gestire.** Calcolo, database e rete sono servizi gestiti.
- **R10 — Protezione dei dati.** Database non raggiungibile da Internet, nessuna password nel codice, permessi minimi per ogni componente, cifratura in transito e a riposo, email degli utenti in un solo posto.
- **R11 — Disponibilità.** Il sistema sopravvive alla caduta di una zona di disponibilità, e nessuna notifica si perde in silenzio.
- **R12 — Dati in Italia.** Dati e servizi in una regione italiana, con latenza bassa per gli utenti.
- **R13 — Operabilità.** Log, metriche e allarmi avvisano il gruppo quando qualcosa non va.

2. La soluzione in sintesi

2.1 L'idea e il diagramma

La piattaforma è **serverless**: non ci sono server nostri, e ogni pezzo è un servizio gestito da AWS che si paga per uso e scala da sé. Il frontend è una SPA React servita da CloudFront; il backend è un'HTTP API di API Gateway che invoca funzioni Lambda; i dati stanno in RDS PostgreSQL, i file in S3, gli utenti in Cognito. Il database vive in una **rete privata senza accesso a Internet**, raggiunta solo dalle Lambda che ne hanno bisogno. Ciò che l'utente non deve aspettare, come l'invio di un'email o la registrazione di un file caricato, viaggia in modo **asincrono**, con ritentativi.

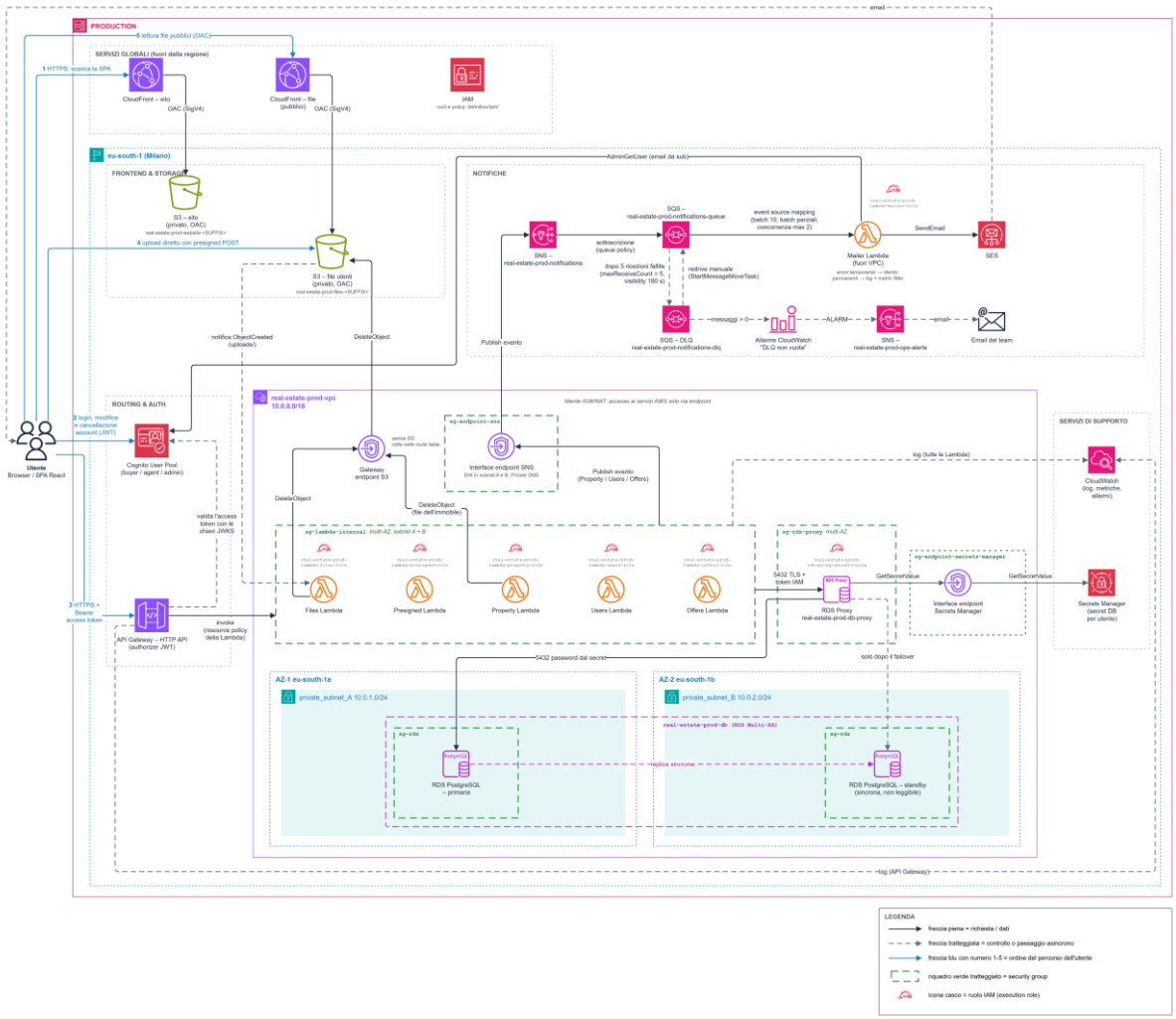


Figura 2.1 — Il diagramma completo dell'infrastruttura.

Una freccia piena è una richiesta o un passaggio di dati, una tratteggiata un controllo o un passaggio asincrono. Le frecce blu numerate seguono l'utente: (1) scarica la SPA, (2) fa login e gestisce l'account, (3) chiama l'API con l'access token, (4) carica un file, (5) legge i file pubblici. Il riquadro verde tratteggiato è un security group, il casco un ruolo IAM. Conviene leggere la figura in tre zone, perché dove sta un servizio decide come lo si protegge.

- **In alto, il mondo esterno e i servizi globali**: il browser, le due distribuzioni CloudFront e IAM.
- **Al centro, i servizi regionali fuori dalla VPC**: Cognito, API Gateway, S3, SNS, SQS, SES, CloudWatch, Secrets Manager e la Mailer Lambda. Non hanno interfacce di rete nostre, quindi niente security group: li protegge solo IAM.

- **In basso, la VPC**: il database, l'RDS Proxy, le cinque Lambda di business e gli endpoint VPC, su due sottoreti private in due zone. Qui valgono sia i security group sia IAM.

Un esempio: un'offerta

Seguiamo un `buyer` che fa un'offerta. Il browser ha già la SPA da CloudFront e un access token da Cognito. Manda la richiesta all'API: l'authorizer JWT verifica il token e API Gateway invoca la Offers Lambda. La Lambda legge il gruppo dai claim e accetta solo un `buyer`; poi entra nell'RDS Proxy con un token IAM e scrive l'offerta nel database. Poi pubblica un evento su SNS, attraverso l'endpoint VPC, e risponde subito al browser. L'email all'agente parte dopo, per conto suo: SNS consegna l'evento alla coda SQS, la Mailer lo legge, chiede a Cognito l'indirizzo dell'agente e invia con SES. Chi autorizza ciascun passaggio è il tema del capitolo 3, flusso per flusso, e del 4.3.

2.2 Dai requisiti alla soluzione

La tabella riprende i requisiti del capitolo 1 e dice come li risolviamo; l'ultima colonna rimanda al flusso che lo mostra nel dettaglio.

Req.	Come lo risolviamo	Servizi (flusso)
R1	la SPA e i file escono da CloudFront, che legge bucket privati; le route di lettura degli annunci non richiedono il token	CloudFront, S3, API Gateway (3.1, 3.4)
R2	un user pool con i gruppi `buyer`, `agent`, `admin`; il browser manda l'access token e un authorizer JWT lo verifica senza codice nostro	Cognito, API Gateway (3.2)
R3	una Lambda per gli immobili: il ruolo arriva nel token e lo controlla la Lambda, la proprietà dell'immobile la dice il database	API Gateway, Lambda, RDS Proxy, RDS PostgreSQL (3.2, 3.3)
R4	il browser carica direttamente su S3 con un presigned POST firmato da una Lambda, che fissa chiave, tipo e dimensione; una notifica di S3 registra il file nel database	Lambda, S3, CloudFront (3.4)
R5	una Lambda per le offerte, con lo stato nel database; ogni passaggio pubblica un evento	Lambda, RDS PostgreSQL, SNS (3.2, 3.3)
R6	l'evento porta solo il `sub` del destinatario; passa da un topic a una coda, e la Mailer chiede l'email a Cognito e invia	SNS, SQS, Lambda, Cognito, SES (3.5)
R7	profilo e cancellazione vanno dal browser a Cognito; la Users Lambda anonimizza il profilo e ritira le offerte in sospeso, avvisando l'altra parte	Cognito, Lambda, RDS PostgreSQL, SNS (3.6)
R8	Lambda e HTTP API si pagano per richiesta e scalano da zero; il proxy protegge il database dai picchi di connessioni; throttling sull'API	Lambda, API Gateway, RDS Proxy (3.2, 3.3)
R9	solo servizi gestiti: nessuna macchina virtuale da amministrare	tutti
R10	VPC senza porte verso Internet; un ruolo IAM e un utente PostgreSQL per Lambda; password solo in Secrets Manager, lette dal proxy; TLS ovunque, cifratura a riposo con KMS; email solo in Cognito	VPC, IAM, RDS Proxy, Secrets Manager, KMS (3.3, 4.1, 4.2)
R11	database Multi-AZ con standby sincrona, Lambda e proxy su entrambe; le email fallite si ritentano e finiscono in una dead-letter queue sorvegliata	RDS PostgreSQL, VPC, SQS (3.3, 3.5)
R12	tutto nella regione `eu-south-1` (Milano)	tutti
R13	log di Lambda e API, metriche dei servizi, allarmi che scrivono al gruppo via email	CloudWatch, SNS (3.7)

2.3 Le scelte di fondo

Le decisioni di dettaglio, con le alternative scartate, sono nel capitolo 5. Qui le quattro che danno forma all'intera architettura.

Serverless

Il traffico è basso e irregolare, e non vogliamo server da gestire (R8, R9). Lambda e un'HTTP API si pagano per richiesta e scalano da zero; un'applicazione sempre accesa costerebbe anche a vuoto. Il prezzo è il cold start sulle istanze nuove e un codice effimero, che non tiene nulla in memoria tra una richiesta e l'altra: lo stato sta nel database e in S3.

Una regione, due zone

Tutto è in `eu-south-1`, per tenere i dati in Italia (R12). Dentro la regione la VPC ha due sottoreti in due zone di disponibilità: il database e il proxy lo richiedono, e se una zona cade RDS promuove la standby dell'altra (R11). Non c'è replica su una seconda regione: è un limite dichiarato nel capitolo 5.

Una VPC senza Internet Gateway né NAT

Nella VPC sta solo ciò che deve stare vicino al database. Il traffico degli utenti entra da CloudFront e API Gateway, che sono fuori, e il backend non chiama API esterne: un NAT sarebbe un costo fisso per niente. I pochi servizi AWS necessari si raggiungono con gli **endpoint VPC**; la Mailer, che non usa il database, sta fuori (4.2).

Sincrono e asincrono

Le richieste degli utenti sono sincrone: il browser chiama l'API e la Lambda risponde. Ciò che può aspettare è asincrono, così un servizio lento non rallenta l'utente e un errore non si perde. Le email passano da SNS e SQS, con ritentativi e una dead-letter queue (3.5); la registrazione di un file parte da una notifica di S3 e non da una conferma del browser, che potrebbe non arrivare o mentire (3.4).

3. Come comunicano i componenti

Il diagramma del capitolo 2 mostra tutte le frecce insieme. Qui lo scomponiamo in sette flussi: per ciascuno seguiamo una richiesta da un capo all'altro e diciamo, a ogni passaggio, **chi parla con chi** e **chi lo autorizza**. I dettagli dei servizi restano solo dove spiegano un collegamento.

Lo schema comune

Ogni flusso si apre con il problema e la soluzione, poi la figura, il percorso, la tabella dei collegamenti, le autorizzazioni chiave, cosa succede quando qualcosa va storto e le scelte. I concetti IAM generali sono nel capitolo 4, le alternative scartate nel 5, la configurazione completa nel repository.

I sette flussi

Sezione	Flusso	In una riga
3.1	Scaricare l'applicazione	CloudFront legge il bucket privato del sito con OAC
3.2	Login e chiamate all'API	Cognito emette i token, l'authorizer JWT controlla l'access token
3.3	Dalla Lambda al database	RDS Proxy con autenticazione IAM verso RDS PostgreSQL
3.4	Caricare e servire i file	presigned POST verso S3, lettura tramite CloudFront
3.5	Le notifiche email	SNS, SQS, Mailer e SES, con la dead-letter queue
3.6	Gestione e cancellazione dell'account	la SPA con Cognito, i dati anonimizzati
3.7	Log, metriche e allarmi	CloudWatch avvisa il team tramite SNS

3.1 Scaricare l'applicazione

Prima di fare login o chiamare l'API il browser deve ricevere la SPA, un insieme di file statici. Il problema è servirli in HTTPS e vicino all'utente senza aprire il bucket a Internet. La soluzione: i file stanno in un **bucket S3 privato** e l'unica porta d'ingresso è una distribuzione **CloudFront**, che legge il bucket con **Origin Access Control** (OAC).

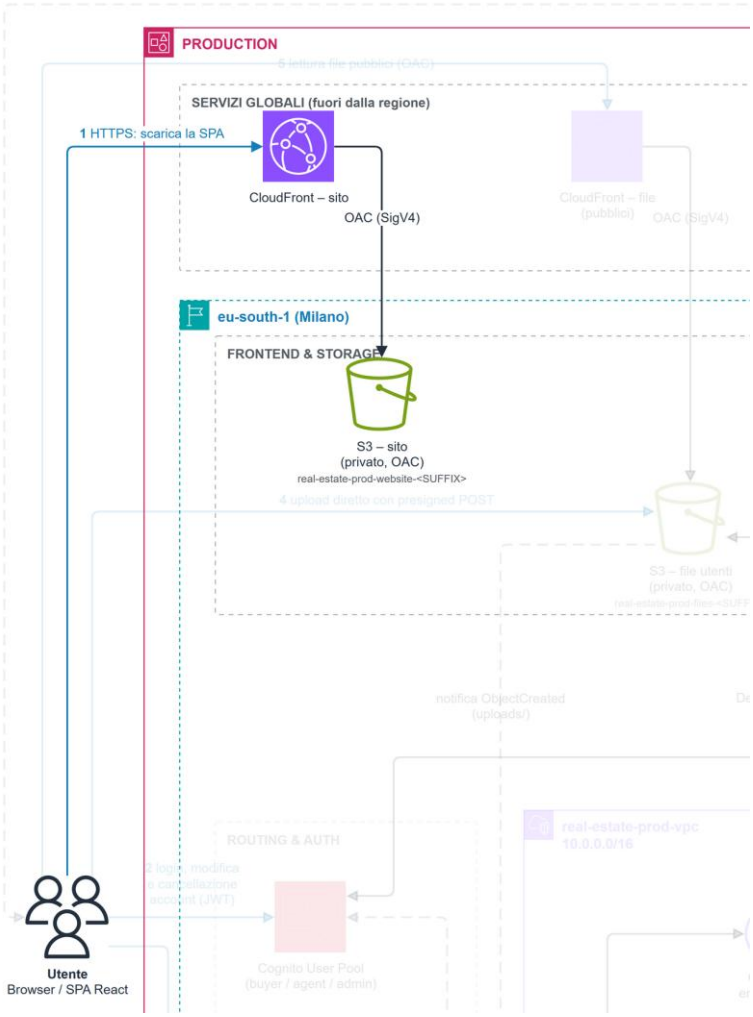


Figura 3.1 — Il download della SPA. Gli elementi che non partecipano sono sbiaditi.

Il percorso

1. Il browser chiede il sito in HTTPS alla distribuzione; una richiesta HTTP viene reindirizzata.
2. Se l'edge non ha il file in cache, lo chiede al bucket `real-estate-prod-website-<SUFFIX>` firmando la richiesta con OAC.
3. S3 valuta la bucket policy e ammette la lettura solo dalla nostra distribuzione.
4. Un percorso della SPA che non è un file fa rispondere S3 con 403: CloudFront restituisce `/index.html` e il routing lo fa il browser.
5. A ogni risposta CloudFront aggiunge gli header di sicurezza.

Chi parla con chi

Da → A	Come	Chi lo autorizza
Browser → distribuzione del sito	HTTPS	nessuno: il sito è pubblico
Distribuzione → bucket del sito	`GetObject` firmato con OAC	bucket policy, con `AWS:SourceArn` = la distribuzione
Distribuzione → browser	risposta con header di sicurezza	response headers policy `real-estate-prod-site-security-headers`

L'autorizzazione chiave

CloudFront **non ha un ruolo IAM**: si presenta come servizio e il permesso sta nella **resource policy del bucket**.

iam/resource-policies/s3-website-bucket-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontSiteDistributionReadOnly",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::real-estate-prod-website-<SUFFIX>/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<DIST_SITE_ID>"
        }
      }
    },
    {
      "Sid": "DenyRequestsWithoutTls",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::real-estate-prod-website-<SUFFIX>",
        "arn:aws:s3:::real-estate-prod-website-<SUFFIX>/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    }
  ]
}
```

Il primo statement concede solo `s3:GetObject`. La condizione `AWS:SourceArn` lo lega alla **nostra** distribuzione: senza, il principal `cloudfront.amazonaws.com` vorrebbe dire qualunque distribuzione di qualunque account (il *confused deputy*, vedi 4.1). Il secondo statement nega le richieste senza TLS. Sul bucket sono attivi anche **Block Public Access** e le ACL disattivate, così un errore futuro in una policy non lo renderebbe pubblico.

Gli header aggiunti da CloudFront difendono i token che la SPA tiene in memoria: la **Content-Security-Policy** ammette solo i nostri script e le chiamate verso API, Cognito e il bucket degli upload. Gli altri header impongono HTTPS (`Strict-Transport-Security`), vietano di incorniciare la pagina (`X-Frame-Options: DENY`) e di indovinare il tipo dei file (`nosniff`).

Quando qualcosa va storto

Un file inesistente dà 403 e non 404, perché CloudFront non ha `s3:ListBucket`; per la SPA va bene, ma per i file degli annunci un file mancante deve restare un errore, ed è il motivo per cui le distribuzioni sono due (3.4). Dopo un deploy si invalida la cache, altrimenti gli edge servono la versione vecchia fino alla scadenza. Un dominio della CSP dimenticato blocca una funzione: va provata con la SPA vera.

Le scelte

Bucket privato con OAC invece di un bucket pubblico con *static website hosting*, che darebbe solo HTTP e nessuna cache. Una CSP nostra, perché la policy gestita da AWS non la include. Nessuna logica sull'edge: il sito è statico e i dati protetti stanno dietro l'API (5.1).

3.2 Login e chiamate all'API

Il problema: come un utente dimostra chi è, e come quella prova arriva alla Lambda che decide se lasciarlo fare. **Cognito** autentica l'utente ed emette i token. L'**HTTP API** di API Gateway controlla il token a ogni richiesta con un **authorizer JWT** nativo. La **Lambda** riceve l'identità già verificata e controlla il ruolo e, dove serve, la proprietà della risorsa.

Il punto delicato è **quale token** va all'API. All'inizio la SPA mandava l'ID token; il docente ci ha fatto notare che l'ID token serve al client per sapere chi è loggato, non ad autorizzare chiamate, e che portava l'email in ogni richiesta. Ora all'API va l'**access token**, come vuole la buona pratica OAuth 2.0 e OpenID Connect; l'**ID token resta nella SPA**.

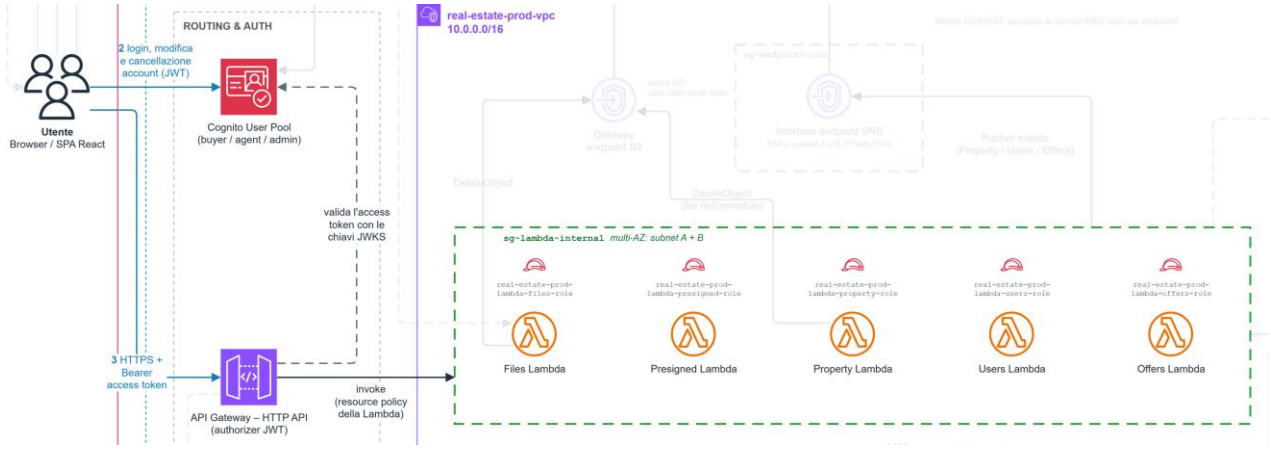


Figura 3.2 — Login con Cognito, chiamata all'API con l'access token, validazione con le chiavi JWKS e invocazione delle Lambda.

Il percorso

1. La SPA manda il browser al **login gestito** di Cognito con il flusso *authorization code* e **PKCE**; l'utente entra con email e password.
2. Cognito torna alla SPA con un codice, che la SPA scambia con ID token, access token e refresh token, tenuti in memoria.
3. Dopo il login la SPA chiama `PUT /users/me`: la Users Lambda crea o aggiorna la riga dell'utente con il solo `sub`.
4. Ogni chiamata all'API porta `Authorization: Bearer <access token>`.
5. L'authorizer JWT verifica firma, issuer, `client_id` e scadenza **senza chiamare Cognito**; se un controllo fallisce risponde 401 e la Lambda non parte.
6. API Gateway invoca la Lambda, che lo ammette con la sua **resource policy**, e le passa i claim del token.
7. La Lambda legge `sub` e gruppi dai claim, controlla il ruolo e, dove serve, la proprietà della risorsa nel database (3.3).

Chi parla con chi

Gli utenti **non sono identità IAM**: IAM compare solo nell'invocazione della Lambda.

Da → A	Come	Chi lo autorizza
SPA → Cognito (login, token)	authorization code con PKCE, HTTPS	configurazione dell'app client; credenziali dell'utente; PKCE al posto di un client secret
SPA → HTTP API	HTTPS con l'access token; preflight CORS	CORS dell'API per il browser; authorizer JWT per il token
Authorizer ← Cognito	chiavi pubbliche JWKS, in cache	nessuno: le chiavi sono pubbliche

Da → A	Come	Chi lo autorizza
HTTP API → Lambda	invocazione sincrona	resource policy della Lambda, con `AWS:SourceArn` = la nostra API
Lambda → database	query di ownership e dati	execution role e `GRANT` (3.3)

I token

L'app client della SPA è **pubblico, senza secret**, perché il codice della SPA lo legge chiunque; PKCE ne prende il posto: solo chi ha iniziato il login può riscattare il codice.

Token	Cosa contiene	A cosa serve
ID token	`sub`, email, gruppi; `aud` = l'app client	resta nella SPA, per sapere chi è loggato
Access token	`sub`, gruppi, `client_id`, scope; **niente email**	va all'API a ogni chiamata e autorizza le operazioni dell'utente su Cognito (3.6)
Refresh token	opaco	rinnova i token; revocato al logout

L'app client `real-estate-prod-spa` ammette solo il flusso `code`, gli scope `openid`, `email` e `aws.cognito.signin.user.admin`, i soli URL di callback e logout del sito, token di 60 minuti e la revoca (configurazione completa nel repository). Lo scope `aws.cognito.signin.user.admin` serve perché l'access token possa agire sull'utente stesso (3.6). I ruoli applicativi non sono scope ma tre **gruppi** del pool, `buyer`, `agent` e `admin`.

L'authorizer JWT

L'authorizer è configurato con l'**issuer**, il nostro user pool, e l'**audience**, l'ID dell'app client. A ogni richiesta controlla la **firma** con le chiavi pubbliche del pool, tenute in cache; l'**issuer**; l'audience, che per l'access token cade su `client_id`, perché il token di Cognito non ha `aud`; e la **scadenza**. Un token falso riceve 401 e non costa un'esecuzione. Le route di sola lettura degli annunci sono pubbliche e non hanno authorizer; tutte le altre sì.

Il CORS non protegge l'API: un client che non è un browser lo ignora. Impedisce solo che un sito di terzi, aperto nel browser dell'utente, la chiami leggendone le risposte; ammette l'origine del sito e i soli metodi e header usati (configurazione nel repository).

Chi può invocare la Lambda

API Gateway non assume un nostro ruolo: è la **funzione** che lo ammette, con una resource policy applicata a tutte e cinque le Lambda di business (`<FUNCTION_NAME>` = `real-estate-prod-property`, `-users`, `-offers`, `-presigned`, `-files`).

iam/resource-policies/lambda-allow-api-gateway.json

```
{
  "Version": "2012-10-17",
  "Id": "default",
  "Statement": [
    {
      "Sid": "AllowHttpApiInvoke",
      "Effect": "Allow",
      "Principal": {
        "Service": "apigateway.amazonaws.com"
      },
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:eu-south-1:<ACCOUNT_ID>:function:<FUNCTION_NAME>",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:execute-api:eu-south-1:<ACCOUNT_ID>:<API_ID>/*"
        }
      }
    }
  ]
}
```

La condizione `AWS:SourceArn` lega il permesso alla nostra API: senza, `apigateway.amazonaws.com` vorrebbe dire qualunque API Gateway, anche di un altro account (confused deputy, vedi 4.1). Due domande, due meccanismi: **chi può chiamare la funzione** lo dice la resource policy, **cosa può fare la funzione** il suo execution role (3.3, 3.5).

Cosa decide la Lambda

La Lambda riceve i claim già validati. Una funzione comune a tutte le Lambda ne legge `sub` e `cognito:groups` e **rifiuta i token con `token_use` diverso da `access`**: l'ID token ha `aud` uguale al nostro client e l'authorizer lo lascerebbe passare. Il `sub` si prende **sempre dai claim**, mai dal body o dai parametri, che il client scrive come vuole.

L'autorizzazione ha due livelli. Il **ruolo** viene dai gruppi: pubblicare un immobile richiede `agent`, fare un'offerta `buyer`. La **proprietà** della risorsa la sa solo il database: la Lambda cerca la riga con quell'id e quel proprietario, e se non la trova (e l'utente non è `admin`) risponde 403. Sulle route che scrivono c'è un ultimo controllo: se la riga dell'utente ha `deleted_at`, l'account è stato cancellato e la scrittura riceve 403 (3.6).

Quando qualcosa va storto

Situazione	Esito
Token mancante o falso, issuer o `client_id` sbagliati, token scaduto	401 dall'authorizer; la SPA rinnova i token
ID token al posto dell'access token	rifiutato dalla funzione comune delle Lambda
Gruppo sbagliato o risorsa di un altro utente	403 dalla Lambda
Scrittura con il token di un account cancellato	403; `PUT /users/me` risponde 410

Furto del token e rischio residuo

Il JWT è firmato, non cifrato: chi lo ha ne legge il payload, che nell'access token è un `sub` opaco, i gruppi e il `client_id`. Il rischio vero è l'uso: è un **bearer token** e vale per chiunque lo presenti. Le difese lo rendono difficile da rubare e lo fanno durare poco: HTTPS ovunque, token in memoria e non in `localStorage`, la CSP del sito (3.1), durata di 60 minuti, revoca del refresh token al logout. L'authorizer però **non vede le revoche**: un access token rubato resta valido fino alla scadenza. Lo accettiamo per un progetto di questa taglia (5.2).

Le scelte

Access token all'API, authorizer JWT nativo invece di un Lambda Authorizer, ruolo dai gruppi e proprietà nella Lambda, login gestito con PKCE. Le alternative scartate sul token (ID token all'API, Pre Token Generation, `/oauth2/userInfo`, Lambda Authorizer) e l'evoluzione verso un Backend For Frontend sono nel capitolo 5.

3.3 Dalla Lambda al database

Cinque Lambda su sei usano il database: Property, Users, Offers, Presigned e Files. Il problema è doppio: ogni istanza di una Lambda apre la propria connessione, e con molte invocazioni concorrenti si arriva al limite di PostgreSQL; e collegarsi richiede credenziali, che non vogliamo nel codice. La soluzione è mettere tra le Lambda e il database `real-estate-prod-db` (Amazon RDS for PostgreSQL) l'**RDS Proxy** `real-estate-prod-db-proxy`, che tiene poche connessioni stabili verso il database e ci smista quelle brevi delle Lambda. I due tratti della catena **non si autenticano allo stesso modo**: le Lambda entrano nel proxy con un token IAM, il proxy entra nel database con la password dell'utente PostgreSQL, che legge da Secrets Manager con un ruolo suo.

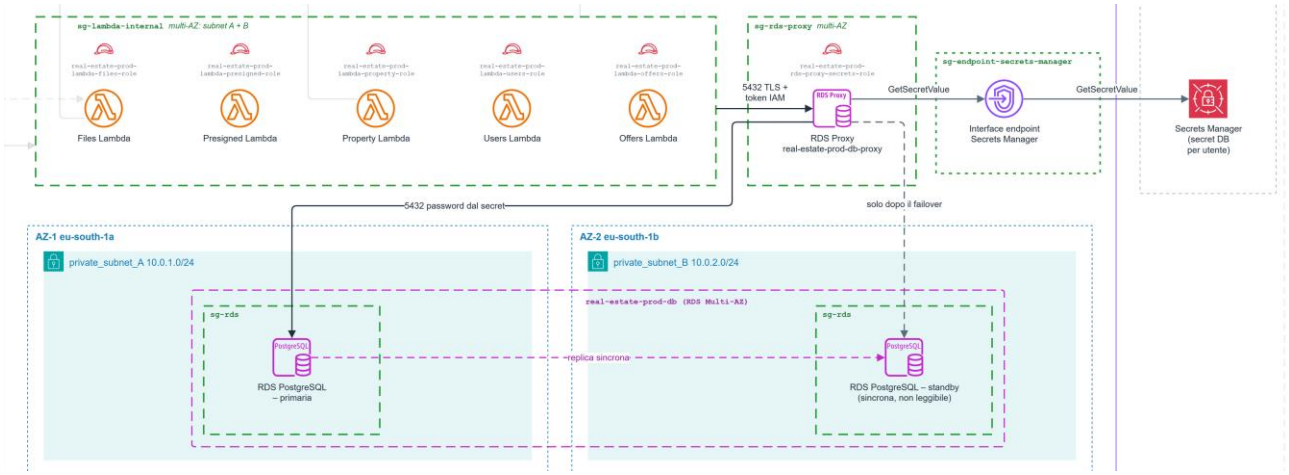


Figura 3.3 — La catena Lambda → RDS Proxy → RDS PostgreSQL, con la lettura dei secret. Gli elementi che non partecipano sono sbiaditi.

Il percorso

1. La Lambda genera con le credenziali temporanee del suo execution role un **token di autenticazione** per il proxy, legato al suo utente PostgreSQL. È una firma calcolata in locale, senza chiamate di rete, e vale 15 minuti.
2. Apre una connessione PostgreSQL verso il proxy sulla porta 5432, su **TLS**, usando il token come password.
3. Il proxy verifica il token con IAM e prende dal pool una connessione verso il database; se deve aprirne una nuova, usa la **password** letta dal secret `real-estate/prod/database/<DB_USER>`.
4. La query arriva all'istanza **primaria**. Da lì decidono i `GRANT` dell'utente PostgreSQL.

Chi parla con chi

Da → A	Come	Chi lo autorizza
Lambda → RDS Proxy	PostgreSQL 5432, TLS obbligatorio, token IAM come password	`rds-db:connect` su una sola coppia proxy/utente nell'execution role
RDS Proxy → Secrets Manager	`GetSecretValue` via HTTPS	ruolo `real-estate-prod-rds-proxy-secrets-role`, assunto dal servizio RDS

Da → A	Come	Chi lo autorizza
Secrets Manager → KMS	decifratura del secret	key policy della chiave gestita `aws/secretsmanager`
RDS Proxy → RDS PostgreSQL	PostgreSQL 5432, TLS, utente e password del secret	credenziali PostgreSQL, poi i `GRANT`

Le autorizzazioni

In questo flusso ci sono **due ruoli** e **due livelli di identità**: il ruolo della Lambda non sa nulla dei secret, il ruolo del proxy non ha `rds-db:connect`.

Entrare nel proxy. Ogni execution role delle cinque Lambda ha un permesso costruito su questo modello:

iam/policies/lambda-connect-proxy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConnectToProxyAsApplicationUser",
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:eu-south-1:<ACCOUNT_ID>:dbuser:<PROXY_RESOURCE_ID>/<DB_USER>"
    }
  ]
}
```

- La risorsa è **una sola coppia** proxy/utente: la Offers può entrare solo come `offers_user`, la Files solo come `files_user`, e così via.
- `<PROXY_RESOURCE_ID>` è l'ID risorsa del proxy, quello che inizia con `prx-`, non il suo nome né il suo ARN.
- Sul proxy sono attive *Require IAM authentication* e *Require TLS*: una Lambda che provasse a entrare con una password, o in chiaro, verrebbe rifiutata.

Leggere il secret. Il proxy non ha credenziali proprie: usa un ruolo che il servizio RDS assume. La trust relationship lo dice:

iam/trust-relationships/rds-proxy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowRdsToAssumeRole",
      "Effect": "Allow",
      "Principal": { "Service": "rds.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Solo il servizio RDS può assumere il ruolo; nessun utente e nessuna Lambda può indossarlo e leggere i secret al posto del proxy. La policy del ruolo permette una sola azione, sui soli secret applicativi:

iam/policies/rds-proxy-read-secrets.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ReadApplicationDatabaseSecrets",
      "Effect": "Allow",
      "Action": "secretsmanager:GetSecretValue",
      "Resource": ["<APPLICATION_SECRET_ARN>"]
    }
  ]
}
```

In `Resource` vanno gli ARN completi dei cinque secret applicativi, con il suffisso casuale che Secrets Manager aggiunge. Il secret dell'utente master non c'è: resta all'amministrazione e non è associato al proxy. Il secret è cifrato con la chiave gestita `aws/secretsmanager`, che Secrets Manager usa da sé quando il proxy lo chiede. Con una chiave del cliente, che oggi non usiamo, il ruolo avrebbe bisogno anche di questa policy (e la key policy dovrebbe ammetterlo):

iam/policies/rds-proxy-decrypt-secrets.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "DecryptSecretsWithCustomerManagedKey",
      "Effect": "Allow",
      "Action": "kms:Decrypt",
      "Resource": "<SECRETS_KMS_KEY_ARN>",
      "Condition": {
        "StringEquals": {
          "kms:ViaService": "secretsmanager.eu-south-1.amazonaws.com"
        }
      }
    }
  ]
}
```

La condizione `kms:ViaService` permette di decifrare solo quando la richiesta arriva da Secrets Manager nella nostra regione: il ruolo non potrebbe usare la chiave chiamando KMS direttamente.

Entrare nel database. Qui IAM non c'è più: il proxy si presenta con utente e password del secret. Per questo agli utenti applicativi non si assegna il ruolo `rds_iam`, che li renderebbe inutilizzabili con la password. IAM decide chi entra nel proxy e con quale utente; i `GRANT` SQL decidono cosa quell'utente può fare sulle tabelle, e ogni Lambda ha il suo utente con i soli `GRANT` che le servono.

L'effetto della separazione è che **la password la vede solo il proxy**. Una Lambda compromessa potrebbe al massimo entrare nel proxy come il suo utente, con i suoi `GRANT`.

In rete i security group formano una catena, `sg-lambda-internal` → `sg-rds-proxy` → `sg-rds` sulla 5432, in cui ognuno accetta solo il precedente: le Lambda non raggiungono il database direttamente (vedi 4.2).

Quando qualcosa va storto

Il database è **Multi-AZ**: la **primaria** in `eu-south-1a` serve le richieste, la **standby** in `eu-south-1b` ne riceve ogni scrittura in modo **sincrono** e non serve letture. Se la primaria cade, RDS promuove la standby sullo stesso endpoint DNS, in genere in uno o due minuti, senza perdere transazioni confermate. Il proxy tiene aperte le connessioni delle Lambda e si ricollega alla nuova primaria: l'interruzione percepita dovrebbe ridursi.

Il motore è **PostgreSQL 18**, la versione principale più recente disponibile su RDS e supportata da RDS Proxy a Milano: rispetto alla 17 ha un anno in più di supporto standard (fino a febbraio 2031), mentre la 19 non è ancora disponibile su RDS.

Gli altri guasti si riconoscono dal punto in cui si fermano. Un timeout all'apertura della connessione è un problema di rete o di security group. Un rifiuto dell'autenticazione è quasi sempre la risorsa di `rds-db:connect` sbagliata. Un target del proxy non disponibile vuol dire che il proxy non riesce a leggere il secret. Un `permission denied` sulle tabelle, a connessione riuscita, vuol dire che IAM ha fatto la sua parte e mancano i `GRANT`.

Il proxy legge il secret chiamando l'API di Secrets Manager dalle subnet private, che non hanno uscita verso Internet: per questo la chiamata passa dall'**interface endpoint** di Secrets Manager. Alcune operazioni SQL, infine, legano una connessione del pool a un solo client (*pinning*) e toglierebbero al proxy il suo vantaggio: il

codice delle Lambda le evita, e l'allarme sulle connessioni del database (3.7) segnala se il pool si avvicina al limite.

Le scelte

Con il traffico del progetto il proxy non è strettamente necessario: lo teniamo perché ci dà il pool condiviso, l'autenticazione IAM e un failover più breve. Tra proxy e database usiamo la password da Secrets Manager e non l'autenticazione IAM end-to-end, perché è la modalità più documentata e lascia la password a un solo componente. Un utente e un secret per Lambda portano il minimo privilegio anche dentro il database. Le alternative scartate sono nel capitolo 5.

3.4 Caricare e servire i file

Un agente pubblica foto e video, e chiunque guardi l'annuncio deve vederli. Far passare i file dall'API non funziona: API Gateway accetta al massimo 10 MB per richiesta e una Lambda sincrona 6 MB, e il tempo di trasferimento lo pagheremmo come esecuzione. La soluzione è che **il file non passi mai dal backend**: la Presigned Lambda non riceve i byte, li autorizza. Firma un *presigned POST*, un modulo di upload firmato, e il browser lo usa per caricare il file direttamente nel bucket privato `real-estate-prod-files-<SUFFIX>`, sotto `uploads/`.

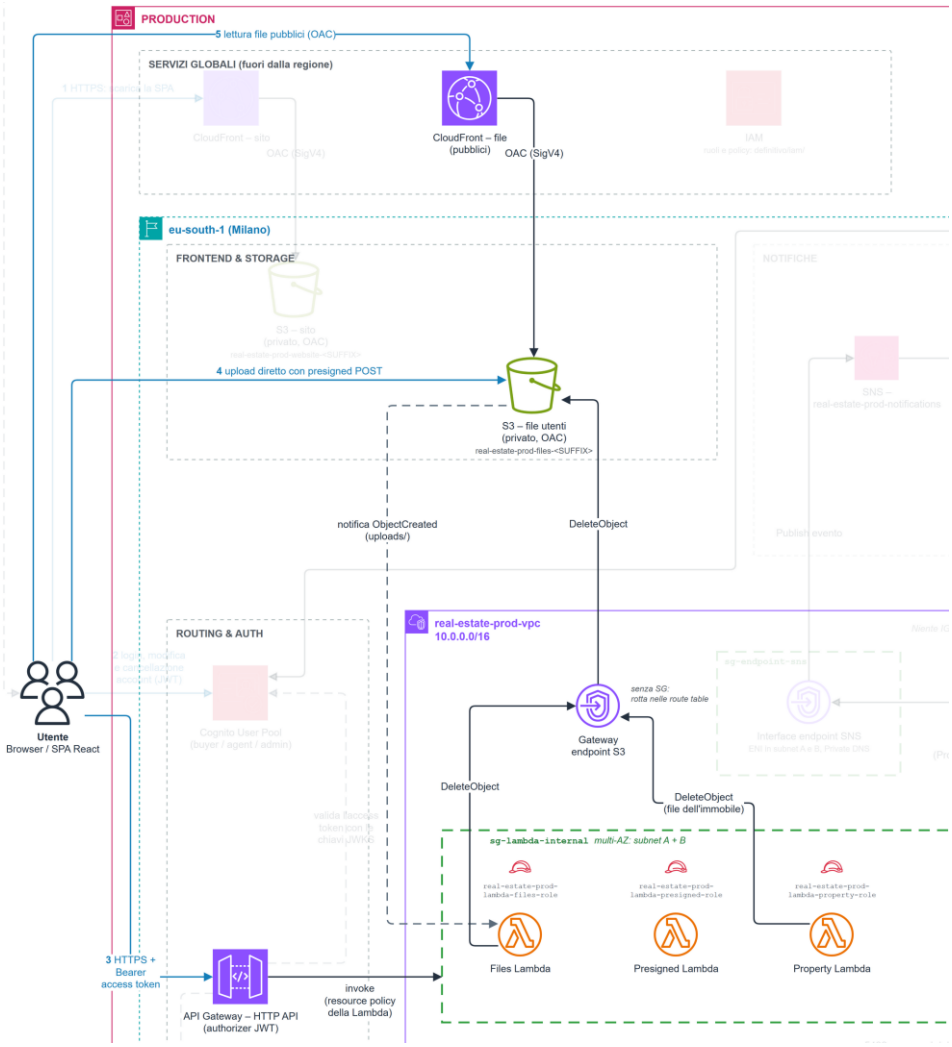


Figura 3.4 — Upload, registrazione, lettura e cancellazione dei file. Gli elementi che non partecipano sono sbiaditi.

Il percorso

1. La SPA chiama `POST /properties/{id}/uploads` con l'access token. La Presigned Lambda controlla nel database che l'utente sia il **proprietario** dell'immobile.
2. La Presigned **firma in locale** un presigned POST: chiave `uploads/<propertyId>/<uuid>.<ext>` decisa da lei, scadenza 5 minuti, tipo `image/` o `video/`, dimensione massima.
3. Il browser invia il file **direttamente al bucket**; lo permette la regola **CORS** del bucket.
4. S3 emette una notifica `ObjectCreated` e **invoca in modo asincrono la Files Lambda**, che registra il file nel database.
5. Per mostrare l'annuncio la SPA scarica i file dalla **distribuzione CloudFront dei file**, che li legge dal bucket con **OAC**.
6. Per cancellare, la Files Lambda (un file) o la Property Lambda (tutti i file di un immobile) chiamano S3 passando dal **gateway endpoint**.

Chi parla con chi

Da → A	Come	Chi lo autorizza
SPA → API → Presigned	`POST /properties/{id}/uploads`, sincrona	authorizer JWT; ownership verificata nel database
Browser → bucket	`POST` del modulo firmato, HTTPS	la firma del ruolo della Presigned e le conditions del modulo; CORS per il browser
S3 → Files Lambda	notifica `ObjectCreated` su `uploads/`, asincrona	resource policy della funzione
CloudFront → bucket	`s3:GetObject` firmato con OAC	bucket policy, solo la distribuzione dei file
Files / Property → bucket	`DeleteObject` (la Property anche `ListBucket`), via gateway endpoint	execution role della Lambda

Le autorizzazioni

Chi firma l'upload. L'utente Cognito non ha permessi IAM e il browser non ha credenziali AWS. Il permesso di scrivere nel bucket ce l'ha il **ruolo della Presigned Lambda**: la funzione usa le sue credenziali temporanee per firmare il modulo, e quando il browser lo invia S3 tratta la richiesta come se arrivasse da quel ruolo.

iam/policies/lambda-presigned.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConnectToProxyAsPresignedUser",
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:eu-south-1:<ACCOUNT_ID>:dbuser:<PROXY_RESOURCE_ID>/presigned_user"
    },
    {
      "Sid": "SignUploadsUnderUploadsPrefix",
      "Effect": "Allow",
      "Action": "s3:PutObject",
      "Resource": "arn:aws:s3:::real-estate-prod-files-<SUFFIX>/uploads/*"
    }
  ]
}
```

- **Un presigned POST ha al massimo i permessi di chi lo firma**: nessun modulo può scrivere fuori da `uploads/*` del bucket dei file.

- **Il permesso si controlla all'upload, non alla firma.** Firmare è un calcolo locale: senza `s3:PutObject` la Lambda firmerebbe senza errori e il browser riceverebbe `AccessDenied`.
- **IAM non limita tipo e dimensione.** Lo fanno le *conditions* del modulo, che la Lambda inserisce nella firma e il client non può cambiare senza invalidarla. Usiamo il POST e non il presigned PUT proprio perché solo il POST permette di limitare la dimensione (le conditions complete sono nel repository).

Il Block Public Access non blocca l'upload: la richiesta è firmata, non pubblica. La regola **CORS** del bucket ammette solo il `POST` e solo dall'origine del sito; come quella dell'API (3.2) protegge l'utente nel browser, non il bucket: l'autorizzazione vera resta la firma.

Chi può invocare la Files Lambda. Qui chi chiama è il servizio S3, non un nostro ruolo, e il permesso sta nella resource policy della funzione:

iam/resource-policies/lambda-files-allow-s3.json

```
{
  "Version": "2012-10-17",
  "Id": "default",
  "Statement": [
    {
      "Sid": "AllowFilesBucketNotifications",
      "Effect": "Allow",
      "Principal": {
        "Service": "s3.amazonaws.com"
      },
      "Action": "lambda:InvokeFunction",
      "Resource": "arn:aws:lambda:eu-south-1:<ACCOUNT_ID>:function:real-estate-prod-files",
      "Condition": {
        "ArnLike": {
          "AWS:SourceArn": "arn:aws:s3:::real-estate-prod-files-<SUFFIX>"
        },
        "StringEquals": {
          "AWS:SourceAccount": "<ACCOUNT_ID>"
        }
      }
    }
  ]
}
```

Le due condizioni servono insieme: `AWS:SourceArn` limita al nostro bucket, `AWS:SourceAccount` al nostro account, perché l'ARN di un bucket non lo contiene (vedi 4.1).

La lettura funziona come per il sito (3.1), con la bucket policy del bucket dei file:

iam/resource-policies/s3-files-bucket-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontFilesDistributionReadOnly",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::real-estate-prod-files-<SUFFIX>/uploads/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::<ACCOUNT_ID>:distribution/<DIST_FILES_ID>"
        }
      }
    },
    {
      "Sid": "DenyRequestsWithoutTls",
      "Effect": "Deny",
      "Principal": "*",
      "Action": "s3:*",
      "Resource": [
        "arn:aws:s3:::real-estate-prod-files-<SUFFIX>",
        "arn:aws:s3:::real-estate-prod-files-<SUFFIX>/*"
      ],
      "Condition": {
        "Bool": {
          "aws:SecureTransport": "false"
        }
      }
    }
  ]
}
```

`s3:GetObject` vale solo su `uploads/*` e solo per la distribuzione dei file, grazie ad `AWS:SourceArn`. Il `Deny` senza TLS vale per tutti, upload firmati compresi. Le distribuzioni sono due perché la regola della SPA "403/404 → `/index.html`" non deve valere per i file.

La cancellazione. Le Lambda nella VPC raggiungono S3 con il **gateway endpoint** (4.2). Il ruolo della Files Lambda, che registra i file e ne cancella uno alla volta:

iam/policies/lambda-files.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConnectToProxyAsFilesUser",
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:eu-south-1:<ACCOUNT_ID>:dbuser:<PROXY_RESOURCE_ID>/files_user"
    },
    {
      "Sid": "DeleteUploadedFiles",
      "Effect": "Allow",
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3:::real-estate-prod-files-<SUFFIX>/uploads/*"
    }
  ]
}
```

Entra nel proxy come `files_user` e può solo cancellare oggetti sotto `uploads/`: non legge, non scrive, non elenca il bucket. La Property, che cancellando un immobile ne svuota il prefisso, ha in più `s3:ListBucket` e il permesso di pubblicare le notifiche (3.5):

iam/policies/lambda-property.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConnectToProxyAsPropertyUser",
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:eu-south-1:<ACCOUNT_ID>:dbuser:<PROXY_RESOURCE_ID>/property_user"
    },
    {
      "Sid": "PublishNotificationEvents",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications"
    },
    {
      "Sid": "ListFilesOfDeletedProperty",
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::real-estate-prod-files-<SUFFIX>",
      "Condition": {
        "StringLike": {
          "s3:prefix": "uploads/*"
        }
      }
    },
    {
      "Sid": "DeleteFilesOfDeletedProperty",
      "Effect": "Allow",
      "Action": "s3:DeleteObject",
      "Resource": "arn:aws:s3:::real-estate-prod-files-<SUFFIX>/uploads/*"
    }
  ]
}
```

`s3:ListBucket` si applica al bucket, non agli oggetti: la condizione `s3:prefix` lo limita a `uploads/*`.

Quando qualcosa va storto

Se l'utente non è il proprietario, la Presigned risponde 403. Un file troppo grande, di tipo sbagliato o inviato dopo i 5 minuti lo rifiuta S3, senza notifica.

La notifica è un'**invocazione asincrona**: S3 non aspetta l'esito. Se la Files fallisce, Lambda ritenta altre due volte, a uno e poi a due minuti di distanza. Uno stesso evento può quindi arrivare più di una volta, e la registrazione deve sopportare il duplicato. Se tutti i tentativi falliscono il file resta su S3 senza riga: non compare nell'annuncio, e la traccia sta nei log e nell'allarme sugli errori della funzione. Un upload firmato poco prima della cancellazione dell'immobile può arrivare dopo: la Files non trova l'immobile e cancella l'oggetto.

Cancellando un immobile, la Property toglie prima la riga e poi i file; se S3 non risponde scrive nel log `FILES_CLEANUP_FAILED`, su cui c'è un allarme (3.7), e la pulizia si rilancia sul prefisso noto. Un file già nella cache di CloudFront può restare servito fino alla scadenza del TTL. Resta un rischio dichiarato: i file non passano da un antivirus, e il tipo controllato è quello dichiarato, non il contenuto.

Le scelte

Registrare il file con la notifica di S3, invece che con una conferma della SPA, fa sì che il database rifletta solo gli upload riusciti: il client potrebbe mentire o non chiamare. La chiave la decide la Lambda, per immobile e con un UUID, così il client non sceglie dove scrivere e la cancellazione lavora su un solo prefisso. Le alternative sono nel capitolo 5.

3.5 Le notifiche email e la dead-letter queue

Quando succede qualcosa che un utente deve sapere (un acquirente fa un'offerta, l'offerta viene accettata o rifiutata, un nuovo utente si registra) il sistema gli manda un'email. Se l'invio avvenisse dentro la richiesta, un servizio di posta lento o fermo farebbe aspettare l'utente e l'email andrebbe persa. Per questo l'invio è **asincrono**: la Lambda che registra l'offerta pubblica un evento e risponde subito; l'email parte dopo, lungo la catena SNS → SQS → Mailer Lambda → SES. La coda permette di ritentare; la **dead-letter queue** (DLQ) raccoglie ciò che fallisce anche dopo i tentativi, e un allarme avvisa il team.

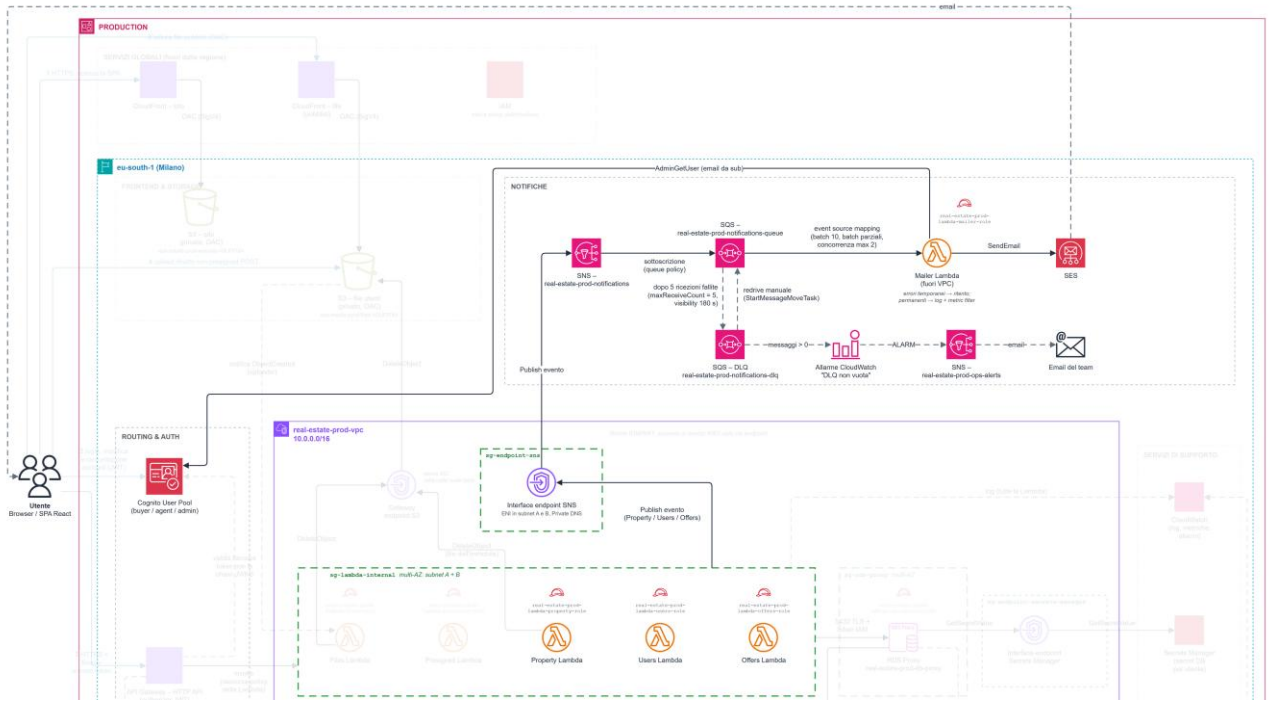


Figura 3.5 — Il flusso delle notifiche, dalla pubblicazione dell'evento alla DLQ e all'allarme. Gli elementi che non partecipano sono sbiaditi.

Il percorso

1. Una Lambda di business (Property, Users o Offers) pubblica un evento sul topic SNS `real-estate-prod-notifications`. Dalla VPC, senza Internet, raggiunge SNS con un **interface endpoint**.
2. SNS consegna l'evento alla coda SQS `real-estate-prod-notifications-queue`, sua unica sottoscrittrice.
3. Il servizio Lambda legge la coda a gruppi di 10 messaggi e invoca la **Mailer Lambda**, al massimo due invocazioni alla volta.
4. Per ogni messaggio la Mailer chiede a **Cognito** l'indirizzo del destinatario e invia l'email con **SES**.
5. Se un invio fallisce per un errore temporaneo il messaggio viene ritentato; dopo 5 tentativi SQS lo sposta nella DLQ `real-estate-prod-notifications-dlq` e un allarme avvisa il team.

L'evento indica il destinatario con il suo `sub`, l'identificativo Cognito che non cambia mai, e non contiene l'indirizzo email: la Mailer lo legge da Cognito al momento dell'invio. Così nessun messaggio in coda, nella DLQ o nei log espone un indirizzo, e se l'utente cambia email la notifica successiva va già a quello nuovo.

Chi parla con chi

Da → A	Come	Chi lo autorizza
Lambda di business → SNS	`sns:Publish` via HTTPS, attraverso l'interface endpoint	execution role della Lambda; endpoint policy; `sg-endpoint-sns` accetta 443 solo da `sg-lambda-internal`
SNS → coda	consegna della sottoscrizione	queue policy, solo dal nostro topic
Coda → Mailer	event source mapping: il servizio Lambda legge e invoca	execution role della Mailer
Mailer → Cognito, SES	`AdminGetUser`, `SendEmail` sugli endpoint pubblici (la Mailer è fuori dalla VPC)	execution role della Mailer
Coda → DLQ	redrive policy, dopo 5 ricezioni	nessun permesso IAM: lo fa SQS
Allarme → topic ops → team	azione dell'allarme CloudWatch, email	stesso account, nessuna policy aggiuntiva

Le autorizzazioni

In questo flusso ci sono i due casi del 4.1: chi chiama è un servizio AWS, e il permesso sta nella resource policy; chi chiama è un nostro ruolo, e sta nella sua identity policy.

Pubblicare sul topic. Chi pubblica è un nostro ruolo. Il ruolo della Offers Lambda, che pubblica gli eventi delle offerte, ha due soli permessi:

iam/policies/lambda-offers.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConnectToProxyAsOffersUser",
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:eu-south-1:<ACCOUNT_ID>:dbuser:<PROXY_RESOURCE_ID>/offers_user"
    },
    {
      "Sid": "PublishNotificationEvents",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications"
    }
  ]
}
```

`sns:Publish` vale su un solo topic. Property e Users hanno lo stesso statement (3.4, 3.6). La pubblicazione passa poi dall'interface endpoint, che ha una sua **endpoint policy**:

iam/resource-policies/vpc-endpoint-sns-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublishToProjectTopicFromThisAccountOnly",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications",
      "Condition": {
        "StringEquals": {
          "aws:PrincipalAccount": "<ACCOUNT_ID>"
        }
      }
    }
  ]
}
```

Non concede nulla: filtra (4.1). Dall'endpoint passa solo `sns:Publish` sul nostro topic da identità del nostro account: una Lambda compromessa non potrebbe usarlo per pubblicare sul topic di qualcun altro, né con credenziali di un altro account.

Entrare nella coda. A scrivere nella coda è il servizio SNS, non un nostro ruolo: il permesso sta nella **queue policy**.

iam/resource-policies/sqs-notifications-queue-policy.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowNotificationsTopicOnly",
      "Effect": "Allow",
      "Principal": {
        "Service": "sns.amazonaws.com"
      },
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications-queue",
      "Condition": {
        "ArnEquals": {
          "aws:SourceArn": "arn:aws:sns:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications"
        }
      }
    }
  ]
}
```

Senza `aws:SourceArn` qualunque topic SNS, anche di un altro account, potrebbe scrivere nella nostra coda (il *confused deputy*, vedi 4.1).

Leggere, cercare l'indirizzo, spedire. Tutto ciò che fa la Mailer è nel suo ruolo:

iam/policies/lambda-mailer.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConsumeNotificationQueue",
      "Effect": "Allow",
      "Action": [
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage",
        "sqs:GetQueueAttributes"
      ],
      "Resource": "arn:aws:sqs:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications-queue"
    },
    {
      "Sid": "ReadRecipientEmailFromUserPool",
      "Effect": "Allow",
      "Action": "cognito-idp:AdminGetUser",
      "Resource": "arn:aws:cognito-idp:eu-south-1:<ACCOUNT_ID>:userpool/<USER_POOL_ID>"
    },
    {
      "Sid": "SendEmailFromProjectAddressOnly",
      "Effect": "Allow",
      "Action": "ses:SendEmail",
      "Resource": "arn:aws:ses:eu-south-1:<ACCOUNT_ID>:identity/*",
      "Condition": {
        "StringEquals": {
          "ses:FromAddress": "<MAIL_FROM>"
        }
      }
    }
  ]
}
```

- A leggere la coda non è il codice della Mailer ma il servizio Lambda, con l'event source mapping; lo fa **con il ruolo della funzione**, per questo i permessi su SQS stanno qui.
- `AdminGetUser` vale su un solo user pool ed è autorizzato con IAM, non con il token di un utente.
- La condizione `ses:FromAddress` impedisce di inviare da un mittente diverso da quello del progetto.

Una Mailer compromessa potrebbe leggere quella coda e gli utenti del pool e inviare email da un solo mittente: nient'altro.

Quando qualcosa va storto: la dead-letter queue

Il ciclo di vita di un messaggio. La coda è di tipo standard: ogni messaggio viene consegnato *almeno una volta*. Quando il servizio Lambda lo riceve per la Mailer, il messaggio diventa invisibile per il *visibility timeout*, 180 secondi. Se l'invio riesce, il messaggio viene cancellato dalla coda. Se fallisce, allo scadere dei 180 secondi torna visibile e viene ricevuto di nuovo. SQS conta le ricezioni: un messaggio già ricevuto 5 volte senza essere cancellato non viene consegnato una sesta, ma spostato nella DLQ. Con cinque tentativi a tre minuti l'uno dall'altro, un'email che continua a fallire arriva nella DLQ dopo circa **15 minuti**.

I due numeri sono legati. I 180 secondi sono sei volte il timeout della Mailer (30 secondi), come raccomanda AWS: un messaggio non torna visibile mentre è ancora in lavorazione. I 5 tentativi sono il minimo raccomandato, perché anche le invocazioni rifiutate per troppa concorrenza contano come ricezioni: con un valore più basso un picco di traffico manderebbe nella DLQ messaggi validi.

Batch parziali. La Mailer riceve fino a 10 messaggi alla volta. Se un solo invio fallisse e la funzione segnalasse un errore, l'intero gruppo tornerebbe in coda: gli altri nove destinatari riceverebbero l'email due volte e consumerebbero tentativi senza motivo. Per questo l'event source mapping accetta i fallimenti parziali: la Mailer indica **solo i messaggi da ritentare**, e gli altri vengono cancellati.

Concorrenza 2. L'event source mapping non lascia lavorare più di due invocazioni della Mailer insieme. Protegge SES, che in sandbox accetta una email al secondo, e Cognito. Se il limite si supera comunque, l'errore di throttling è trattato come temporaneo e il messaggio aspetta il tentativo successivo.

Errori temporanei e permanenti. La Mailer li distingue:

Tipo	Esempi	Esito
Temporaneo	throttling o errore 5xx di SES o Cognito, timeout di rete	ritentato; dopo 5 ricezioni va nella DLQ
Permanente	messaggio malformato, email non verificata, destinatario rifiutato da SES	log di errore e messaggio cancellato; un filtro sui log fa scattare un allarme
Previsto	l'utente ha cancellato l'account e Cognito non lo trova	log informativo, messaggio cancellato senza allarme

Ritentare un errore permanente occuperebbe la coda per un quarto d'ora e finirebbe comunque nella DLQ, confuso con i messaggi da rimandare. Così nella DLQ c'è solo ciò che è fallito per una causa **rimediabile**.

Quanto resta un messaggio. La scadenza si conta dal momento in cui il messaggio è entrato nella coda originale, non nella DLQ. Per questo la coda principale tiene i messaggi 4 giorni e la DLQ 14, il massimo: un messaggio arrivato in DLQ resta analizzabile per almeno una decina di giorni.

Perché la DLQ sta sulla coda e non sulla Lambda. La DLQ di Lambda e le *on-failure destination* valgono per le invocazioni asincrone, come quella di S3 (3.4). Con SQS come sorgente è la coda a contare le ricezioni, e la sua redrive policy decide quando spostare un messaggio. Lo spostamento lo esegue SQS, senza permessi IAM; sulla DLQ una regola ammette come sorgente solo la nostra coda (configurazione nel repository).

L'allarme e cosa fa il team

Un allarme CloudWatch controlla ogni 5 minuti quanti messaggi ci sono nella DLQ e, appena sono più di zero, pubblica sul topic `real-estate-prod-ops-alerts`, a cui è iscritta l'email del team. Un secondo allarme guarda

l'età del messaggio più vecchio nella coda principale: se supera i 15 minuti la Mailer è ferma o troppo lenta, e lo si scopre prima che i messaggi arrivino nella DLQ.

Il redrive dalla DLQ verso la coda principale è **manuale** e viene dopo aver capito la causa: chi è di turno guarda i messaggi, cerca nei log della Mailer perché sono falliti, corregge il problema (una quota di SES, un servizio fermo, un errore nel modello dell'email) e solo allora li rimanda nella coda, a velocità bassa. Ripartono da zero ricezioni; un messaggio da scartare si annota e si cancella. Un redrive automatico ripeterebbe un errore non risolto. Chi esegue la procedura, una persona del team e non un servizio, ha una policy dedicata:

iam/policies/ops-dlq-redrive.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InspectAndRedriveNotificationsDlq",
      "Effect": "Allow",
      "Action": [
        "sqs:ReceiveMessage",
        "sqs:DeleteMessage",
        "sqs:GetQueueAttributes",
        "sqs:StartMessageMoveTask",
        "sqs:ListMessageMoveTasks",
        "sqs:CancelMessageMoveTask",
        "sqs:ListDeadLetterSourceQueues"
      ],
      "Resource": "arn:aws:sqs:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications-dlq"
    },
    {
      "Sid": "SendRedrivenMessagesToNotificationsQueue",
      "Effect": "Allow",
      "Action": "sqs:SendMessage",
      "Resource": "arn:aws:sqs:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications-queue"
    },
    {
      "Sid": "ReadMailerLogsToFindTheCause",
      "Effect": "Allow",
      "Action": "logs:FilterLogEvents",
      "Resource": "arn:aws:logs:eu-south-1:<ACCOUNT_ID>:log-group:/aws/lambda/real-estate-prod-mailer:*"
    }
  ]
}
```

Sulla DLQ ha le azioni per guardare i messaggi e avviare il *move task*; sulla coda principale solo `sqs:SendMessage`, perché il task scrive lì con le credenziali dell'operatore; nei log solo `FilterLogEvents` sul log group della Mailer. Non può cancellare le code né cambiarne la configurazione.

Le scelte

SNS davanti a SQS permette di aggiungere altri canali (SMS, push) senza toccare chi pubblica. La Mailer sta fuori dalla VPC perché non usa il database, e raggiunge Cognito e SES senza endpoint privati. Le alternative sono nel capitolo 5.

3.6 Gestione e cancellazione dell'account

Cambiare email o password e cancellare l'account sono operazioni sull'identità, che sta in Cognito. Le Lambda di business però stanno in una VPC senza accesso a Internet e senza un endpoint per Cognito: non lo raggiungono. La soluzione è che la **SPA parli direttamente con Cognito**, con le API pensate perché l'utente gestisca il proprio account, autorizzate dal suo **access token** invece che da IAM. Alla nostra API resta un solo compito: **anonimizzare** i dati applicativi prima della cancellazione.

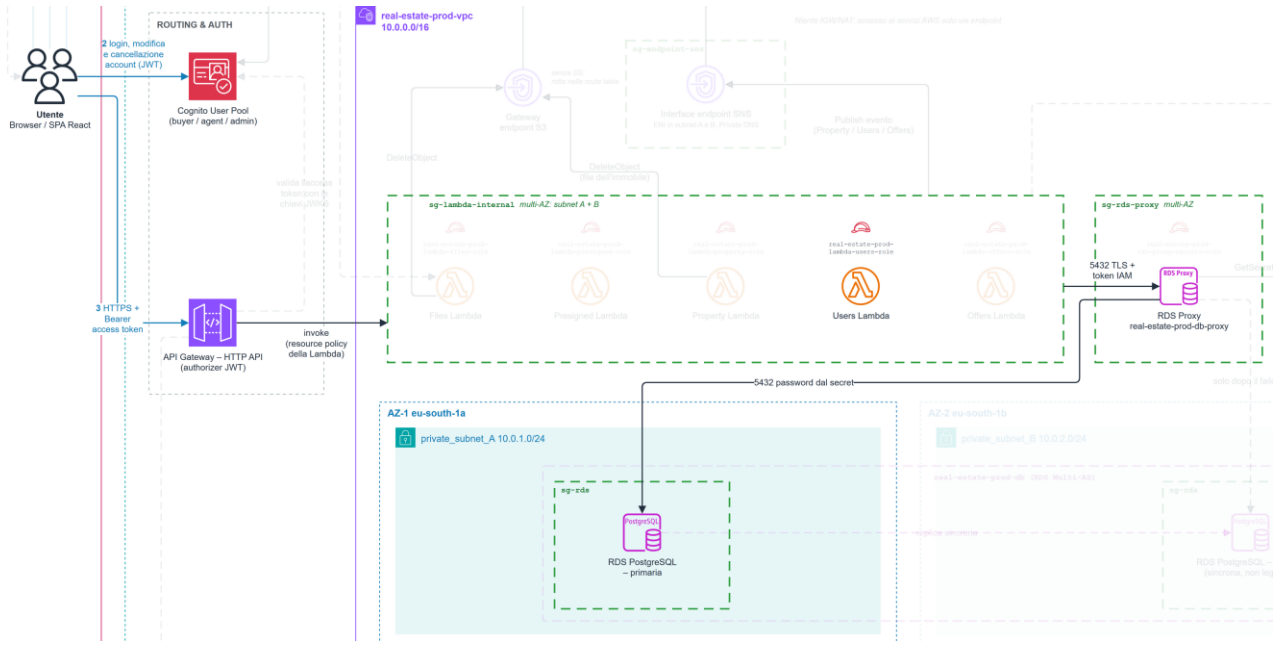


Figura 3.6 — Gestione e cancellazione dell'account: la SPA verso Cognito e verso la Users Lambda, che anonimizza i dati attraverso l'RDS Proxy.

Il percorso

Cambio dell'email: la SPA chiama `UpdateUserAttributes` con l'access token; Cognito manda un codice al nuovo indirizzo e, finché l'utente non lo conferma, resta attiva l'email vecchia. Nel database non c'è niente da aggiornare. **Cancellazione**:

1. **Nuovo login di conferma** con `prompt=login`, che obbliga a ridare la password: `DeleteUser` non la chiede, e un token rubato basterebbe.
2. `DELETE /users/me` sull'API: la Users Lambda anonimizza i dati in una transazione e avvisa l'altra parte delle offerte ritirate.
3. `DeleteUser` su Cognito con lo stesso access token: l'utente sparisce e il refresh token non vale più.
4. **Logout**: la SPA butta i token dalla memoria.

Chi parla con chi

Da → A	Come	Chi lo autorizza
SPA → Cognito	`UpdateUserAttributes`, `ChangePassword`, `DeleteUser`	access token con scope `aws.cognito.signin.user.admin`; nessun permesso IAM
SPA → HTTP API → Users Lambda	`DELETE /users/me`	authorizer JWT e resource policy (3.2)
Users Lambda → RDS Proxy → database	una transazione	`rds-db:connect` come `users_user`; `GRANT`
Users Lambda → SNS	evento `OFFER_WITHDRAWN`	`sns:Publish` sul topic (3.5)

Le autorizzazioni chiave

Cognito ha due famiglie di API. Quelle **dell'utente** ricevono un access token e agiscono solo sul suo titolare: il browser non ha credenziali AWS e il nostro account non gli concede nulla. Quelle **admin** agiscono su qualunque utente e si autorizzano solo con IAM: le usa soltanto la Mailer, in lettura (3.5). Il cambio email lo regola il user pool: l'email è l'attributo di accesso e resta la vecchia finché la nuova non è confermata, così un

errore di battitura o un token rubato non spostano l'account su un indirizzo che l'utente non controlla. La configurazione del user pool è nel repository.

La Users Lambda **non ha permessi su Cognito**. Il suo ruolo è quello della Offers (3.5) con un altro utente del database:

iam/policies/lambda-users.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ConnectToProxyAsUsersUser",
      "Effect": "Allow",
      "Action": "rds-db:connect",
      "Resource": "arn:aws:rds-db:eu-south-1:<ACCOUNT_ID>:dbuser:<PROXY_RESOURCE_ID>/users_user"
    },
    {
      "Sid": "PublishNotificationEvents",
      "Effect": "Allow",
      "Action": "sns:Publish",
      "Resource": "arn:aws:sns:eu-south-1:<ACCOUNT_ID>:real-estate-prod-notifications"
    }
  ]
}
```

Entra nel proxy come `users_user` e pubblica sul topic, niente di più e nessuna azione su Cognito. Il `sub` da cancellare arriva dai claim: non c'è modo di chiederle di cancellare un altro utente. Che cosa può toccare nel database lo decidono i `GRANT`: sugli immobili e sulle offerte può cambiare **solo lo stato**, e non può cancellare righe.

L'ordine e l'anonimizzazione

Database e Cognito non hanno una transazione comune, quindi l'ordine conta. Prima il database, poi Cognito: se `DeleteUser` fallisce, l'account esiste ancora e l'utente può rientrare; `PUT /users/me` risponde **410**, e la SPA capisce che deve solo ripetere `DeleteUser`. Con l'ordine inverso, un fallimento lascerebbe dati che nessuno può più cancellare, perché l'utente non può più fare login.

Nel database **anonimizziamo** invece di cancellare, in una sola transazione. Il profilo viene svuotato e resta il solo `sub`, segnato come cancellato con `deleted_at`. Gli immobili dell'utente vengono ritirati: escono dagli elenchi e non ricevono offerte né upload. Le offerte in sospeso, fatte da lui o ricevute sui suoi immobili, vengono ritirate, e l'altra parte riceve un'email. Le offerte già chiuse restano come storia; i file su S3 restano con l'immobile ritirato. Ripetere la chiamata è innocuo.

Il `sub` che resta non è più un dato personale: l'unico legame con una persona era l'email in Cognito, che sparisce con `DeleteUser`. È la scelta di tenere l'email **solo in Cognito** a rendere l'anonimizzazione così piccola.

Quando qualcosa va storto

L'access token già emesso vale ancora fino a un'ora, e l'authorizer non sa che l'utente non esiste più: per questo le route che scrivono controllano `deleted_at` e rispondono **403** (3.2), al costo di una query sulla chiave primaria per ogni scrittura. Se il browser si chiude tra il passo 2 e il 3 si resta nello stato recuperabile descritto sopra. Una notifica ancora in coda per l'utente cancellato fa rispondere a Cognito che l'utente non esiste: la Mailer la scarta senza allarme (3.5). Se l'avviso all'altra parte non parte, la cancellazione riesce lo stesso. Restano fuori i file degli immobili ritirati, ancora raggiungibili da chi ha il link, e i backup del database fino alla loro scadenza.

Le scelte

Account gestito dalla SPA con le API dell'utente, invece di `AdminDeleteUser` da una Lambda che dalla VPC non arriva a Cognito. Prima il database, poi Cognito. Anonimizzare invece di cancellare a cascata, perché un'offerta riguarda anche l'altra parte (5.1).

3.7 Log, metriche e allarmi

Con soli servizi gestiti non c'è un server su cui collegarsi a guardare cosa succede: l'osservabilità va progettata. Usiamo **CloudWatch** per raccogliere i log delle Lambda e dell'HTTP API, leggere le metriche che i servizi pubblicano da soli e far scattare pochi **allarmi**, che pubblicano sul topic SNS `real-estate-prod-ops-alerts`, iscritto all'email del team. È un topic separato da quello delle notifiche agli utenti: altro pubblico, altro pubblicatore.

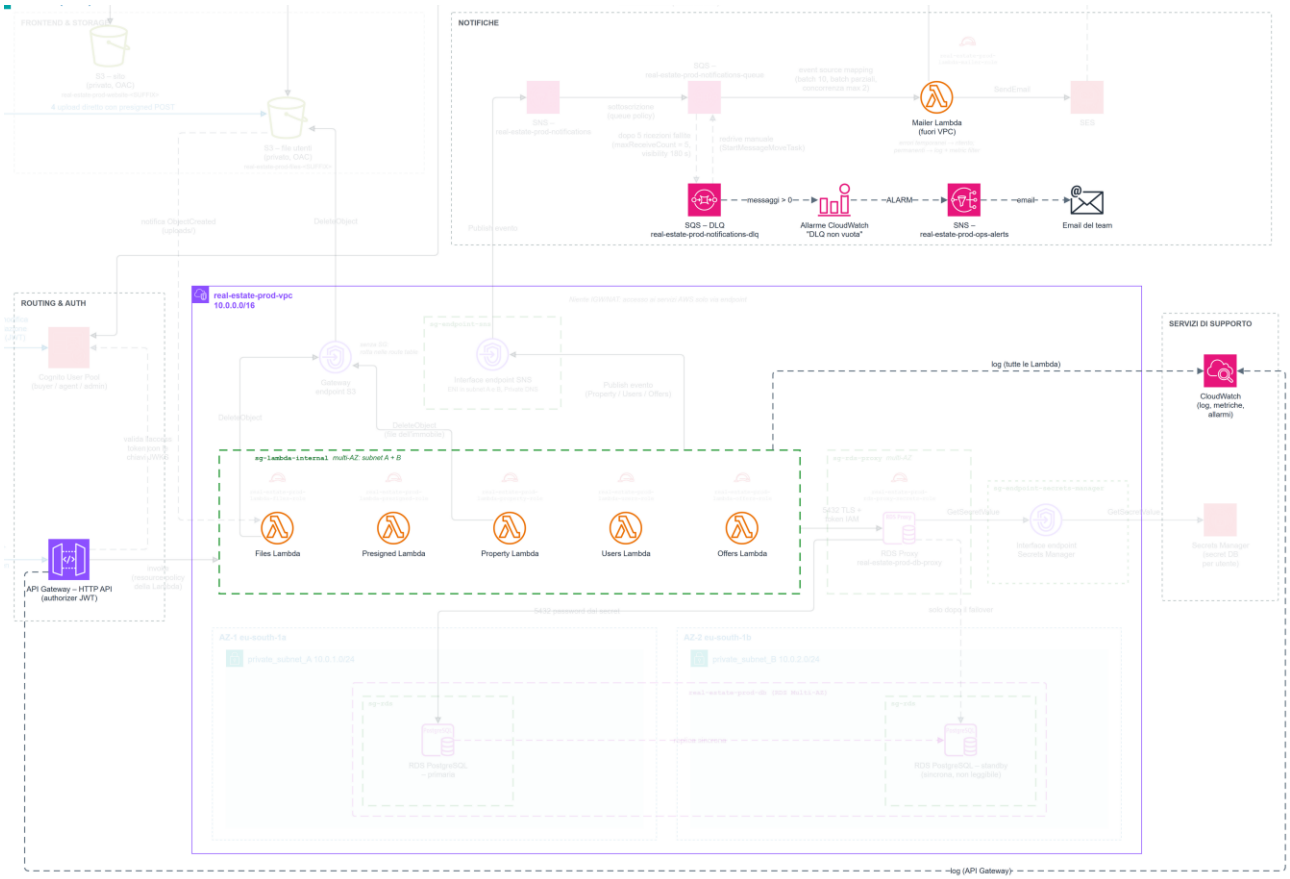


Figura 3.7 — Log, metriche e allarmi. Gli elementi che non partecipano sono sbiaditi.

Il percorso

1. Ogni Lambda scrive nel log group `/aws/lambda/<nome-funzione>`; lo stage dell'HTTP API scrive il suo access log.
2. I servizi pubblicano le loro metriche: errori delle Lambda, CPU e connessioni del database, età dei messaggi nelle code.
3. Per ciò che esiste solo nei log, un **metric filter** trasforma le righe che corrispondono a un pattern in una metrica.
4. Un allarme che supera la soglia pubblica sul topic, che consegna l'email al team.

Chi parla con chi

Da → A	Come	Chi lo autorizza
Lambda → CloudWatch Logs	lo invia il servizio Lambda, non la rete della funzione	policy gestita `AWSLambdaBasicExecutionRole`
HTTP API → CloudWatch Logs	access log dello stage	nessun ruolo: basta indicare il log group
Allarme → topic ops	azione dell'allarme	nessuna topic policy: stesso account, nessuna chiave KMS del cliente
Operatore → log della Mailer	`logs:FilterLogEvents`	policy `ops-dlq-redrive` (3.5), sul solo log group della Mailer

Le autorizzazioni chiave

Le Lambda nella VPC non hanno Internet, eppure scrivono i log senza un endpoint: li raccoglie il servizio Lambda. Il permesso di scrittura sta in una delle due policy gestite che usiamo; per i log la sua `Resource: "*"` è accettabile, mentre per gli altri servizi ogni ruolo ha una policy nostra sull'ARN preciso (4.1). L'allarme non ha un ruolo: con un topic nello stesso account e non cifrato con una nostra chiave KMS, la procedura standard funziona senza topic policy.

Un errore **permanente** della Mailer non viene ritentato e non arriva mai nella dead-letter queue: senza un controllo, l'email andrebbe persa in silenzio. La Mailer scrive log in JSON, senza indirizzi email, e un metric filter sul suo log group conta le righe con `level` uguale a `ERROR` e `code` uguale a `MAIL_PERMANENT_FAILURE` nella metrica `MailerPermanentFailures`, su cui c'è un allarme (configurazione completa nel repository).

Gli allarmi

Allarme	Cosa significa
Messaggi nella DLQ	un'email non è partita dopo i tentativi (3.5)
Messaggi vecchi in coda	la Mailer è ferma; lo si vede prima della DLQ
Errori permanenti della Mailer	email scartate senza ritentare
Errori delle Lambda	una funzione sta fallendo
CPU e connessioni del database	il database è sotto pressione: CPU oltre l'80% o connessioni oltre l'80% del massimo, per 15 minuti
Pulizia dei file non riuscita	cancellato un immobile, dei file sono rimasti su S3 (3.4)

Quando qualcosa va storto

Se nessuno conferma la sottoscrizione email, gli allarmi non arrivano: è la prima cosa da verificare. Gli allarmi coprono il backend, non l'esperienza nel browser.

Le scelte

Pochi allarmi, ciascuno legato a un'azione umana, verso un solo topic separato. Nessun endpoint VPC per i log, che costerebbe senza servire.

4. Sicurezza e IAM

Nei flussi ogni freccia è una richiesta, e ogni richiesta è stata permessa **due volte**: dalla rete, che decide se i pacchetti arrivano, e da IAM, che decide se chi chiama può fare quell'operazione su quella risorsa. Qui mettiamo in ordine le regole che il capitolo 3 ha applicato caso per caso. Gli utenti finali (`buyer`, `agent`, `admin`) non sono identità IAM: li autenticano Cognito e l'authorizer JWT, e la Lambda controlla gruppo e ownership (3.2). Qui parliamo dei servizi.

4.1 Le tre facce di IAM

IAM risponde a una domanda: **chi** può fare **quale azione** su **quale risorsa**, a **quali condizioni**. Di default la risposta è no. Nessun componente ha credenziali proprie: le Lambda e l'RDS Proxy usano **ruoli**, identità senza password che il servizio assume ricevendo da STS credenziali temporanee. Nel codice e nel repository non c'è nessuna chiave.

Ogni permesso del progetto sta in uno di tre documenti, con la stessa grammatica (`Effect`, `Action`, `Resource`, `Condition`) ma attaccati a cose diverse:

- **Identity policy**, su un ruolo: *cosa può fare* chi lo indossa. Esempio: `lambda-offers.json` concede `rds-db:connect` come `offers_user` e `sns:Publish` sul topic (3.5).
- **Resource policy**, su una risorsa (funzione, bucket, coda, endpoint): *chi può usarla*. Esempio: la queue policy ammette `sns.amazonaws.com` dal nostro topic.
- **Trust relationship**, su un ruolo: *chi può assumerlo*. È la resource policy del ruolo.

L'**execution role** di una Lambda le mette insieme. La trust, uguale per i sei ruoli delle Lambda, permette al servizio Lambda di assumerlo; le identity policy dicono cosa fa il codice; chi può invocare la funzione sta invece nella resource policy della funzione. Il ruolo del proxy ha la stessa struttura con `rds.amazonaws.com` (3.3).

iam/trust-relationships/lambda.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowLambdaToAssumeRole",
      "Effect": "Allow",
      "Principal": { "Service": "lambda.amazonaws.com" },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

Come si valuta una richiesta

AWS raccoglie tutte le policy che riguardano la richiesta e decide così: un `Deny` esplicito vince su tutto; altrimenti serve almeno un `Allow`; altrimenti è no. **Nello stesso account** basta che l'`Allow` stia nella identity policy **oppure** nella resource policy. Fanno eccezione le trust e le key policy di KMS, che devono ammettere il principal comunque. Il `Deny` esplicito del progetto è nelle due bucket policy, che rifiutano ogni richiesta senza TLS, anche dai nostri ruoli.

Per questo molte frecce hanno solo la identity policy: la Files Lambda cancella oggetti, la Mailer legge la coda, il proxy legge i secret, e nessuna di queste risorse nomina il ruolo. Quando invece chi chiama è **un servizio AWS** (API Gateway, S3, CloudFront, SNS), il suo principal non ha identity policy nel nostro account: l'unico posto per il suo `Allow` è la resource policy della risorsa che chiama. È il caso delle cinque Lambda di business, della Files Lambda, dei due bucket e della coda.

Per la stessa eccezione la coda è cifrata con SSE-SQS e non con `aws/sqs`: la key policy di quella chiave non ammette `sns.amazonaws.com` e non si può modificare. L'**endpoint policy** infine non concede nulla: filtra. Una richiesta che passa dall'endpoint SNS deve essere permessa dall'endpoint **e** dal ruolo (testo in 3.5).

Confused deputy

Ammettere `sns.amazonaws.com` vuol dire ammettere il servizio intero, cioè i topic di tutti i clienti AWS: un altro account potrebbe usare SNS come intermediario per scrivere nella nostra coda. È il **confused deputy**. Ogni nostra resource policy verso un servizio ha quindi `aws:SourceArn`, che il servizio valorizza con la risorsa per cui agisce: la nostra API, il nostro bucket, la nostra distribuzione, il nostro topic. Solo verso S3 aggiungiamo `aws:SourceAccount`, perché gli ARN di S3 non contengono l'account e un bucket cancellato potrebbe essere ricreato da altri con lo stesso nome (la policy della Files Lambda è in 3.4).

Privilegio minimo e iam:PassRole

- **Un ruolo per Lambda**: se la Offers fosse compromessa non potrebbe cancellare file né leggere la coda, e CloudTrail dice quale funzione ha fatto cosa.
- **Un utente PostgreSQL per Lambda**: `rds-db:connect` vale per una sola coppia proxy/utente, poi i `GRANT` limitano le tabelle (3.3).
- **Policy gestite da AWS solo per log e interfacce di rete**; tutto il resto in policy nostre, su ARN precisi.
- **Condizioni che stringono**: `ses:FromAddress` per il mittente, `s3:prefix` per l'elenco della Property, `kms:ViaService` per un'eventuale chiave del cliente.

Nessun ruolo ha `s3:GetObject` (i file li legge solo CloudFront) e nessuna Lambda ha `secretsmanager:GetSecretValue` (la password la vede solo il proxy).

Chi decide quale ruolo indossa una funzione è chi la crea, e per farlo serve `iam:PassRole` su quel ruolo. Senza questo controllo un'identità con pochi permessi potrebbe creare una Lambda, consegnarle un ruolo potente e ottenerne i permessi tramite il codice. I permessi di provisioning stanno solo nell'identità di deploy del team, non nei ruoli applicativi, e il `PassRole` è limitato così:

iam/policies/deploy-pass-role.json

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PassProjectRolesToLambdaAndRdsOnly",
      "Effect": "Allow",
      "Action": "iam:PassRole",
      "Resource": "arn:aws:iam::<ACCOUNT_ID>:role/real-estate-prod-*",
      "Condition": {
        "StringEquals": {
          "iam:PassedToService": [
            "lambda.amazonaws.com",
            "rds.amazonaws.com"
          ]
        }
      }
    }
  ]
}
```

La risorsa ammette solo i ruoli `real-estate-prod-*`, e la condizione `iam:PassedToService` solo Lambda e RDS: un ruolo del progetto non può finire su un'istanza EC2 o su un altro servizio, e nessun ruolo di altri può essere consegnato.

4.2 La rete come secondo livello

IAM decide chi può fare cosa; la rete decide **chi può raggiungere chi**. Dentro la VPC valgono tutti e due; fuori la rete non è nostra e resta solo IAM.

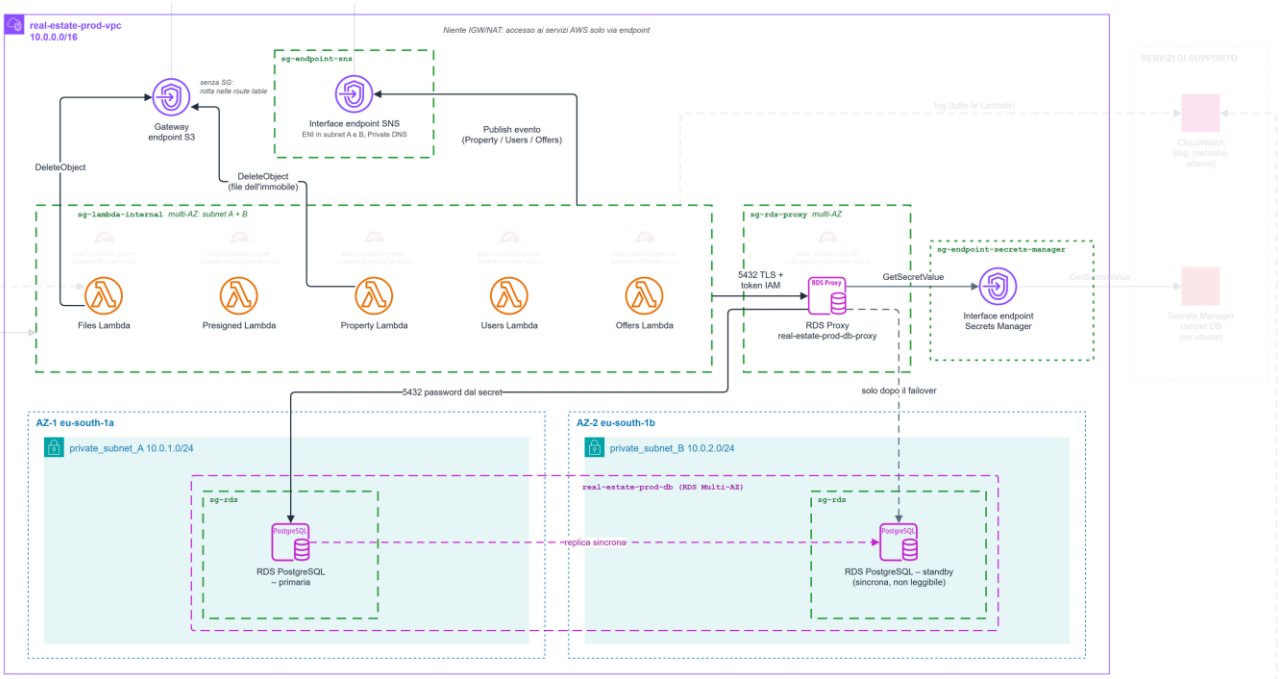


Figura 4.1 — La VPC con le due subnet private, i security group e gli endpoint.

`real-estate-prod-vpc` non ha Internet Gateway né NAT: le due subnet private, `10.0.1.0/24` in `eu-south-1a` e `10.0.2.0/24` in `eu-south-1b`, hanno solo la rotta locale e quella verso il gateway S3. Dentro ci sono solo il database, l'RDS Proxy, le interfacce di rete delle cinque Lambda di business e gli interface endpoint; perché niente NAT e perché due zone lo spiega il 2.3.

I security group usano come sorgente **altri security group**, non indirizzi IP, e formano una catena:

Security group	In entrata	In uscita
`sg-lambda-internal` (5 Lambda)	nessuna	5432 → proxy; 443 → endpoint SNS; 443 → prefix list S3
`sg-rds-proxy`	5432 da Lambda	5432 → database; 443 → endpoint Secrets Manager
`sg-rds` (database)	5432 dal proxy	nessuna
`sg-endpoint-sns`	443 da Lambda	nessuna
`sg-endpoint-secrets-manager`	443 dal proxy	nessuna

Le Lambda non raggiungono il database direttamente, nessuno entra nelle Lambda, e ogni endpoint accetta solo chi lo usa.

Senza Internet i servizi AWS si raggiungono dagli **endpoint VPC**, di due tipi. Il **gateway** di S3 è una rotta verso la prefix list, gratuita e senza security group. Gli **interface** di SNS e Secrets Manager sono interfacce di rete con IP privato in ogni subnet, a pagamento, con Private DNS: l'SDK usa il nome di sempre. L'endpoint SNS ha un'**endpoint policy** che lascia passare solo `sns:Publish` sul nostro topic da identità del nostro account (3.5); gli altri usano quella di default. Tutto il resto, Mailer compresa, sta fuori dalla VPC e si protegge solo con IAM.

4.3 La mappa delle autorizzazioni

Tutti i passaggi del sistema, per flusso: su quale rete viaggiano e che cosa li autorizza. Le policy di ogni gruppo sono nel paragrafo che lo intitola; la trust delle Lambda e `iam:PassRole` sono in 4.1.

Da → A	Rete	Autorizzato da
3.1 Scaricare l'applicazione		
Browser → CloudFront del sito	Internet, HTTPS	nessuno: contenuto pubblico
CloudFront → bucket del sito	AWS, firma OAC	bucket policy con `aws:SourceArn`; Deny senza TLS
3.2 Login e chiamate all'API		
Browser → Cognito	Internet, HTTPS	credenziali dell'utente, code con PKCE
Browser → HTTP API	Internet, HTTPS	CORS; authorizer JWT
HTTP API → 5 Lambda	AWS	resource policy della funzione con `aws:SourceArn`
Servizio Lambda → execution role	STS	trust relationship (4.1)
3.3 Dalla Lambda al database		
5 Lambda → RDS Proxy	VPC, 5432 TLS	`rds-db:connect` nel ruolo
Servizio RDS → ruolo del proxy	STS	trust relationship
Proxy → Secrets Manager	VPC, endpoint 443	`GetSecretValue` nel ruolo del proxy
Proxy → database	VPC, 5432 TLS	password PostgreSQL, poi `GRANT`
3.4 Caricare e servire i file		
Browser → bucket, upload	Internet, HTTPS	presigned POST firmato col ruolo della Presigned; CORS
S3 → Files Lambda	AWS	resource policy con `aws:SourceArn` e `aws:SourceAccount`
Browser → CloudFront → bucket	Internet; AWS con OAC	bucket policy, solo `uploads/*`
Files, Property → S3, cancellazione	VPC, gateway S3	`DeleteObject` nel ruolo
3.5 Notifiche e DLQ		
Property, Users, Offers → SNS	VPC, endpoint 443	`sns:Publish` nel ruolo; endpoint policy
SNS → coda	AWS	queue policy con `aws:SourceArn`
Coda → Mailer	AWS	lettura della coda nel ruolo della Mailer
Mailer → Cognito, SES	endpoint regionali	`AdminGetUser`, `SendEmail` nel ruolo
Coda → DLQ; redrive	SQS; console	`RedrivePolicy`; policy dell'operatore
3.6 Gestione dell'account		
Browser → Cognito	Internet, HTTPS	access token dell'utente
SPA → `/users/me`	come 3.2	authorizer JWT, `sub` dei claim
3.7 Log, metriche e allarmi		
Lambda → CloudWatch Logs	servizio Lambda	`AWSLambdaBasicExecutionRole`
Allarme → topic degli allarmi	AWS, poi email	stesso account: nessuna topic policy

Ne escono tre gruppi: le frecce che partono dalla VPC hanno **rete e IAM**; quelle tra servizi gestiti **solo IAM**; quelle del browser non passano da IAM, e le autorizzano Cognito, l'authorizer JWT, la firma del presigned POST e la Lambda.

5. Decisioni, alternative e limiti

Qui raccogliamo le decisioni che hanno dato forma al progetto, con l'alternativa lasciata, poi i limiti che dichiariamo e gli sviluppi possibili.

5.1 Le decisioni principali

Decisione	Perché	Alternativa scartata
Architettura serverless	traffico basso e variabile: si paga per richiesta, scala da zero, nessun server da gestire	container su ECS Fargate, sempre accesi
Regione `eu-south-1`	dati in Italia, latenza bassa; tutti i servizi usati ci sono	Francoforte o Irlanda: dati fuori dall'Italia
VPC senza Internet Gateway né NAT, con endpoint VPC	il traffico utente entra da servizi fuori dalla VPC e non chiamiamo API esterne	NAT Gateway: costo fisso orario senza necessità
HTTP API con authorizer JWT nativo	costa meno; verifica firma, issuer, `client_id` e scadenza senza codice nostro	REST API; Lambda Authorizer, una funzione in più a ogni richiesta
Access token all'API; la Mailer legge l'email da Cognito con `AdminGetUser`	l'ID token serve a dire al client chi è l'utente, non ad autorizzare le API; così nessuna email sta nei token, negli eventi o nel database	ID token all'API (la nostra prima scelta, poi rivista con il docente); una Lambda Pre Token Generation che aggiunge l'email all'access token; l'endpoint `userInfo`, non raggiungibile dalla VPC senza NAT; una cache nelle Lambda; un Lambda Authorizer
Upload con presigned POST e registrazione da notifica S3	il file non passa dal backend; solo il POST limita la dimensione; il database vede solo gli upload riusciti	
SPA e file via CloudFront, bucket privati con OAC	HTTPS e cache; i file degli annunci li vede chi ha il link, ma mai dal bucket	bucket pubblico con *static website hosting*; URL firmati, utili solo per documenti privati
RDS Proxy con token IAM, un utente PostgreSQL e un secret per Lambda	nessuna password nelle Lambda; IAM decide chi entra, i `GRANT` cosa fa	connessione diretta al database
RDS PostgreSQL Multi-AZ, primaria e standby sincrona (indicazione del docente)	failover automatico se cade una zona; costo minore, gestione più semplice	Aurora con writer e reader (prima scelta): la reader serviva solo al failover; Multi-AZ DB cluster, più costoso; Single-AZ, senza HA
SNS → SQS → Mailer, DLQ dopo 5 ricezioni, visibility timeout 180 s	buffer e ritentativi; SNS permette altri consumer; i valori seguono le raccomandazioni AWS	EventBridge; solo SQS; DLQ sulla Lambda, che vale solo per invocazioni asincrone
Alla cancellazione dell'account **anonimizziamo**	resta il solo `sub`; offerte e immobili passano a `withdrawn`, l'altra parte viene avvisata e le chiavi esterne restano valide	cancellazione a cascata, che toglie la storia all'altra parte
Un ruolo IAM per Lambda; `aws:SourceArn` nelle resource policy	permessi minimi e audit chiaro; niente *confused deputy*	policy gestite larghe; principal di servizio senza condizioni

5.2 Limiti e rischi dichiarati

- **Configurazione mai provata nell'insieme.** La relazione descrive la configurazione prevista: non tuttele risorse sono state create. Probabilmente durante l'esposizione l'architettura sarà online ma non lo è ancora in data 24.10.2026.
- **Revoca dei token.** L'authorizer JWT non vede le revoche: un access token rubato resta valido fino alla scadenza, al massimo un'ora, anche dopo il logout (3.2). Dopo la cancellazione dell'account può solo leggere (3.6).
- **Costo fisso.** Il database, il proxy e gli interface endpoint costano anche a vuoto: è la parte che non segue R8.
- **Una sola regione.** Le due zone coprono la caduta di un data center, non quella della regione.
- **Failover non istantaneo.** Per uno o due minuti le scritture si fermano.
- **Email doppie, raramente.** La coda standard consegna almeno una volta; evitarlo richiederebbe un database nella Mailer.
- **Cosa resta di un account cancellato.** La riga con il solo `sub`, gli immobili ritirati con i loro file, il profilo nei backup del database fino alla fine della retention (3.6).
- **File e sicurezza dei contenuti.** Foto e video sono leggibili da chi ha il link, e nessun antivirus analizza i file caricati: restano solo i limiti di tipo e dimensione.

5.3 Sviluppi possibili

- Un **Backend For Frontend** che tenga i token e dia al browser un cookie `HttpOnly`: risolve il furto del token.
- **WAF** su una distribuzione CloudFront davanti all'API, perché non si associa direttamente a un'HTTP API.
- **Scansione antivirus** dei file, **pulizia dei file** degli immobili ritirati, URL firmati per eventuali **documenti privati**, un **dominio personalizzato**.